JAVA REFRESHER

Chapters 2·9 | Complete Collection

CONTENTS

Click any chapter to jump directly to its first page.

02	Java Language Refresher	2
03	Inheritance, Abstract Classes & Interfaces (Focused)	26
04	Essential Java Types	35
05	Collections & Generics (Expanded)	75
06	Exceptions & Resource Management	117
07	Lambdas & Functional Interfaces	139
08	Streams & Optional	154
09	Annotations, Reflection & Generated Code	190

The chapter files are preserved without reformatting.

JAVA LANGUAGE REFRESHER

CHAPTER 02 · VARIABLES TO LOOPS

Deep refresher · 8–9/10 target · Java 17+

> COLOR LEGEND purple = keywords and operators blue = numbers orange = strings teal = comments

38 illustrated Java examples with verified output beneath each. Download includes 8 combined, topic-based Java programs for IntelliJ IDEA.

Focus: variables, all primitive ranges, numeric conversions, String, operators, decisions, and loops. Arrays and OOP explanations are saved for later chapters.

YOUR LEARNING MAP

1. Variables, identifiers and scope
2. Primitive data types and ranges
3. Conversions, promotion and casting
4. Strings in depth
5. Operators and expression evaluation
6. Decisions: if, else and switch
7. Loops: repetition and control
8. Synthesis: predicting output and fixing bugs
9. Mastery checks and reference

1. Variables, identifiers and scope

A variable is a named location for a value. Java checks its type at compile time. Declarations tell Java what type a variable holds; assignments give it a value.

> Core mental model Declaration: int count; Assignment: count = 4; Initialization: int count = 4; Reassignment changes a variable, not its type. Local variables require assignment before reading; fields receive defaults (covered in OOP).

Declare, initialize and reassign · Example 01

Notice that Java is statically typed; type is not inferred again when the value changes.

```
/**
 * CHAPTER 2 | Declare, initialize and reassign
 * WHY IT MATTERS: Notice that Java is statically typed; type is not inferred again
 * when the value changes.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex01Variables {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int age = 46;                   // int stores whole numbers
        double price = 19.95;           // decimal values
        boolean active = true;          // true or false
        char grade = 'A';               // one UTF-16 code unit
        // STEP 2: Compare the output with your prediction.
        System.out.println(age + " | " + price + " | " + active + " | " + grade);
        age = 47;                       // reassignment, still int
        System.out.println("Next age: " + age);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
46 | 19.95 | true | A
Next age: 47
```

Identifiers, naming and final · Example 02

Java names are case-sensitive. Prefer lowerCamelCase for variables and UPPER_SNAKE_CASE for constants.

```
/**
 * CHAPTER 2 | Identifiers, naming and final
 * WHY IT MATTERS: Java names are case-sensitive. Prefer lowerCamelCase for variables
 * and UPPER_SNAKE_CASE for constants.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex02NamesAndConstants {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int customerCount = 20;         // descriptive lowerCamelCase
        int customerCOUNT = 3;          // a DIFFERENT name
        final double SALES_TAX_RATE = 0.0825;
        double subtotal = 100;
        // STEP 2: Compare the output with your prediction.
        System.out.println(customerCount + customerCOUNT);
        System.out.println(subtotal * SALES_TAX_RATE);
        // SALES_TAX_RATE = 0.09;       // compile error: final variable
```

```
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
23
8.25
```

Block scope and shadowing rules · Example 03

Local variables exist only within the block where declared. You cannot redeclare the same local name in an overlapping block.

```
/**
 * CHAPTER 2 | Block scope and shadowing rules
 * WHY IT MATTERS: Local variables exist only within the block where declared. You
 * cannot redeclare the same local name in an overlapping block.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex03Scope {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int outer = 10;
        if (outer > 0) {
            int inner = 20;            // only visible inside this block
            // STEP 2: Compare the output with your prediction.
        System.out.println(outer + inner);
        }
        // System.out.println(inner);   // compile error: out of scope
        System.out.println(outer);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
30
10
```

Local variable type inference with var · Example 04

Since Java 10, var asks the compiler to infer the type from the initializer. It is still statically typed; it is not a dynamic variable.

```
/**
 * CHAPTER 2 | Local variable type inference with var
 * WHY IT MATTERS: Since Java 10, var asks the compiler to infer the type from the
 * initializer. It is still statically typed; it is not a dynamic variable.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex04VarInference {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        var count = 10;                 // inferred int
        var message = "Hello";         // inferred String
        var rate = 2.5;                 // inferred double
        // count = "ten";               // compile error
        // var missing;                  // compile error, no initializer
        // STEP 2: Compare the output with your prediction.
        System.out.println(message + " " + count * rate);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Hello 25.0
```

> **Identifier rules** Names can contain letters, digits, underscore, and dollar sign, but cannot start with a digit. Keywords (e.g., class, int, if) are disallowed; _ alone is reserved. Avoid dollar signs and confusing Unicode identifiers in ordinary code.

2. Primitive data types and ranges

Java has exactly eight primitive types: byte, short, int, long, float, double, char and boolean. String is NOT a primitive; it is a reference type.

Type	Size	Range / meaning	Literal hint
byte	8 bits	-128 to 127	(byte) 12
short	16 bits	-32,768 to 32,767	(short) 12
int	32 bits	-2,147,483,648 to 2,147,483,647	123
long	64 bits	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	123L
float	32 bits	~6-7 significant decimal digits; finite magnitude ~1.4E-45 to 3.4E38	1.2F
double	64 bits	~15-16 significant decimal digits; finite magnitude ~4.9E-324 to 1.8E308	1.2 (default)
char	16 bits	0 to 65,535 UTF-16 code units (\u0000 to \uFFFF)	'A'
boolean	JVM-dependent	true or false; no specified storage size	true

> **Precision is not decimal accuracy** float and double store binary floating-point approximations. double is the usual choice for non-financial scientific/measurement calculations. For exact decimal money, use BigDecimal (introduced briefly here; later chapter can cover APIs).

Print official integer and character ranges · Example 05

MIN_VALUE and MAX_VALUE are the built-in constants for numeric primitive wrapper types.

```
/**
 * CHAPTER 2 | Print official integer and character ranges
 * WHY IT MATTERS: MIN_VALUE and MAX_VALUE are the built-in constants for numeric
 * primitive wrapper types.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex05PrimitiveRanges {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        // STEP 2: Compare the output with your prediction.
        System.out.println("byte:   " + Byte.MIN_VALUE + " .. " + Byte.MAX_VALUE);
        System.out.println("short:  " + Short.MIN_VALUE + " .. " + Short.MAX_VALUE);
        System.out.println("int:    " + Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
```

```
        System.out.println("long:   " + Long.MIN_VALUE + " .. " + Long.MAX_VALUE);
        System.out.println("char numeric: " + (int) Character.MIN_VALUE + " .. " + (int) Character.MAX_VALUE);
        System.out.println("float max:  " + Float.MAX_VALUE);
        System.out.println("double max: " + Double.MAX_VALUE);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
byte:   -128 .. 127
short:  -32768 .. 32767
int:    -2147483648 .. 2147483647
long:   -9223372036854775808 .. 9223372036854775807
char numeric: 0 .. 65535
float max:  3.4028235E38
double max: 1.7976931348623157E308
```

Numeric literals, suffixes and readability · Example 06

The suffix L makes long, F makes float; scientific notation and numeric underscores improve readability.

```
/**
 * CHAPTER 2 | Numeric literals, suffixes and readability
 * WHY IT MATTERS: The suffix L makes long, F makes float; scientific notation and
 * numeric underscores improve readability.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex06Literals {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        long population = 8_200_000_000L; // L mandatory for large long literal
        float temperature = 36.6F;      // without F, 36.6 is double
        double scientific = 6.02e23;
        int binary = 0b1010;            // binary = 10
        int hex = 0xFF;                  // hex = 255
        int octal = 012;                 // octal = 10; avoid leading zeros
        // STEP 2: Compare the output with your prediction.
        System.out.println(population + " " + temperature);
        System.out.println(scientific + " " + binary + " " + hex + " " + octal);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
8200000000 36.6
6.02E23 10 255 10
```

Integer overflow and exact arithmetic · Example 07

Overflow silently wraps around for ordinary integer arithmetic. Math.addExact detects it by throwing ArithmeticException.

```
/**
 * CHAPTER 2 | Integer overflow and exact arithmetic
 * WHY IT MATTERS: Overflow silently wraps around for ordinary integer arithmetic.
 * Math.addExact detects it by throwing ArithmeticException.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex07Overflow {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int max = Integer.MAX_VALUE;
        // STEP 2: Compare the output with your prediction.
        System.out.println(max + 1);    // wraps to Integer.MIN_VALUE
```

```
        try {
            System.out.println(Math.addExact(max, 1));
        } catch (ArithmeticException e) {
            System.out.println("Overflow detected");
        }
        long safer = (long) max + 1;    // promote BEFORE addition
        System.out.println(safer);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
-2147483648
Overflow detected
2147483648
```

Floating-point approximation and special values · Example 08

Never use == for approximate decimal results. Infinity and NaN are valid floating-point values, not exceptions.

```
/**
 * CHAPTER 2 | Floating-point approximation and special values
 * WHY IT MATTERS: Never use == for approximate decimal results. Infinity and NaN are
 * valid floating-point values, not exceptions.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex08FloatingPoint {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        double result = 0.1 + 0.2;
        // STEP 2: Compare the output with your prediction.
        System.out.println(result);     // commonly 0.30000000000000004
        System.out.println(result == 0.3); // false
        System.out.println(Math.abs(result - 0.3) < 1e-9); // tolerance check
        System.out.println(1.0 / 0.0); // Infinity
        System.out.println(0.0 / 0.0); // NaN
        System.out.println(Double.isNaN(0.0 / 0.0));
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
0.30000000000000004
false
true
Infinity
NaN
true
```

char, escapes and Unicode · Example 09

char is one UTF-16 code unit, not necessarily a complete Unicode character; supplementary symbols use two code units.

```
/**
 * CHAPTER 2 | char, escapes and Unicode
 * WHY IT MATTERS: char is one UTF-16 code unit, not necessarily a complete Unicode
 * character; supplementary symbols use two code units.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex09CharUnicode {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        char letter = 'A';
```

```
        char devanagari = '\u0905';   // Unicode escape for Devanagari A
        char newline = '\n';
        // STEP 2: Compare the output with your prediction.
        System.out.println(letter + " numeric code unit " + (int) letter);
        System.out.println(devanagari);
        System.out.print("First line" + newline + "Second line\n");
        String emoji = "";          // often two UTF-16 code units
        System.out.println("Emoji length in UTF-16 units: " + emoji.length());
        System.out.println("Emoji code points: " + emoji.codePointCount(0, emoji.length()));
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
A numeric code unit 65
□
First line
Second line
Emoji length in UTF-16 units: 2
Emoji code points: 1
```

3. Conversions, promotion and casting

Widening converts a narrower compatible numeric type to a wider one automatically; narrowing needs an explicit cast and can lose information. Beware: widening int/long to float/double can lose precision.

Widening vs narrowing · Example 10

A cast tells Java you accept possible truncation/wrap. It does not magically clamp the value into range.

```
/**
 * CHAPTER 2 | Widening vs narrowing
 * WHY IT MATTERS: A cast tells Java you accept possible truncation/wrap. It does not
 * magically clamp the value into range.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex10WideningNarrowing {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int small = 100;
        long wide = small;               // implicit widening
        double decimal = wide;          // implicit widening
        double amount = 12.99;
        int truncated = (int) amount;    // 12, NOT rounded
        int large = 130;
        byte wrapped = (byte) large;     // -126, not 127
        // STEP 2: Compare the output with your prediction.
        System.out.println(wide + " / " + decimal);
        System.out.println(truncated + " / " + wrapped);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
100 / 100.0
12 / -126
```

Arithmetic promotions and integer division · Example 11

byte and short operands normally promote to int. int / int truncates toward zero, even when assigned to double.

```java
/**
 * CHAPTER 2 | Arithmetic promotions and integer division
 * WHY IT MATTERS: byte and short operands normally promote to int. int / int truncates
 * toward zero, even when assigned to double.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex11Promotion {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        byte a = 10;
        byte b = 20;
        int sum = a + b;                // expression promoted to int
        // STEP 2: Compare the output with your prediction.
        System.out.println(sum);
        System.out.println(5 / 2);      // 2
        System.out.println(5 / 2.0);    // 2.5
        double wrong = 5 / 2;           // evaluated as int BEFORE assignment
        double correct = (double) 5 / 2;
        System.out.println(wrong + " vs " + correct);
        System.out.println(-5 / 2);     // -2, truncates toward zero
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
30
2
2.5
2.0 vs 2.5
-2
```

When widening loses precision · Example 12

A very large long cannot always be represented exactly by a double; numeric range and precision are different.

```java
/**
 * CHAPTER 2 | When widening loses precision
 * WHY IT MATTERS: A very large long cannot always be represented exactly by a double;
 * numeric range and precision are different.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex12PrecisionLong {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        long exact = 9_007_199_254_740_993L; // 2^53 + 1
        double approximation = exact;
        // STEP 2: Compare the output with your prediction.
        System.out.println(exact);
        System.out.printf("%.0f%n", approximation);
        System.out.println((long) approximation == exact); // false
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
9007199254740993
9007199254740992
false
```

4. Strings in depth

A String is an immutable sequence of UTF-16 code units. Create it using double quotes; characters use single quotes. String methods typically return NEW values rather than changing the original.

String creation, length and character access · Example 13

Indexes begin at zero; charAt returns a char code unit. Do not treat length() as a human-character count for every Unicode string.

```
/**
 * CHAPTER 2 | String creation, length and character access
 * WHY IT MATTERS: Indexes begin at zero; charAt returns a char code unit. Do not treat
 * length() as a human-character count for every Unicode string.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex13StringBasics {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String firstName = "Java";
        String empty = "";
        // STEP 2: Compare the output with your prediction.
        System.out.println(firstName.length());
        System.out.println(firstName.charAt(0));    // J
        System.out.println(firstName.charAt(firstName.length() - 1)); // a
        System.out.println(empty.isEmpty());
        System.out.println("  ".isBlank());         // Java 11+
        // firstName.charAt(4);                     // runtime exception
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
4
J
a
true
true
```

substring, search, replace and trimming · Example 14

substring(start,end) includes start and excludes end. String methods return a new result.

```
/**
 * CHAPTER 2 | substring, search, replace and trimming
 * WHY IT MATTERS: substring(start,end) includes start and excludes end. String methods
 * return a new result.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex14StringMethods {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String input = "  Learn Java Quickly  ";
        String cleaned = input.strip();             // Unicode-aware Java 11+
        // STEP 2: Compare the output with your prediction.
        System.out.println(cleaned.toUpperCase());
        System.out.println(cleaned.substring(6, 10)); // Java
        System.out.println(cleaned.indexOf("Java"));
        System.out.println(cleaned.contains("Quick"));
        System.out.println(cleaned.replace("Quickly", "Well"));
        System.out.println(input.length());         // input unchanged
```

```
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
LEARN JAVA QUICKLY
Java
6
true
Learn Java Well
22
```

equals vs == and string pool · Example 15

== tests whether references identify the same object; equals checks String content. Interner optimizations can make == appear to work.

```
/**
 * CHAPTER 2 | equals vs == and string pool
 * WHY IT MATTERS: == tests whether references identify the same object; equals checks
 * String content. Interner optimizations can make == appear to work.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex15StringEquality {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String a = "hello";
        String b = "hello";
        String c = new String("hello");
        // STEP 2: Compare the output with your prediction.
        System.out.println(a == b);       // often true: pooled literals
        System.out.println(a == c);       // false: c is a new object
        System.out.println(a.equals(c));  // true: same content
        System.out.println("HELLO".equalsIgnoreCase(a));
        System.out.println("hello".equals(c)); // literal-first null safe
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
true
false
true
true
true
```

null, blank, empty and safe checks · Example 16

null means no object reference. The empty string is an existing String with length zero. isBlank detects whitespace-only text.

```
/**
 * CHAPTER 2 | null, blank, empty and safe checks
 * WHY IT MATTERS: null means no object reference. The empty string is an existing
 * String with length zero. isBlank detects whitespace-only text.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex16StringNull {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String absent = null;
        String empty = "";
        String spaces = "  ";
```

```
        // STEP 2: Compare the output with your prediction.
        System.out.println(absent == null);     // true
        System.out.println(empty.isEmpty());    // true
        System.out.println(spaces.isBlank());   // true
        System.out.println("ok".equals(absent)); // false, safe
        // absent.length();                      // NullPointerException
        if (absent != null && !absent.isBlank()) {
            System.out.println(absent);
        } else {
            System.out.println("No nonblank value");
        }
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
true
true
true
false
No nonblank value
```

Immutability and concatenation · Example 17

toUpperCase and replace cannot change the original String. += builds another String; repeated concatenation in a loop may be inefficient.

```
/**
 * CHAPTER 2 | Immutability and concatenation
 * WHY IT MATTERS: toUpperCase and replace cannot change the original String. += builds
 * another String; repeated concatenation in a loop may be inefficient.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex17StringImmutability {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String name = "java";
        String uppercase = name.toUpperCase();
        // STEP 2: Compare the output with your prediction.
        System.out.println(name);         // still java
        System.out.println(uppercase);    // JAVA
        name += " language";             // name now refers to another String
        System.out.println(name);
        StringBuilder builder = new StringBuilder();
        for (int n = 1; n <= 3; n++) builder.append(n);
        System.out.println(builder.toString());
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
java
JAVA
java language
123
```

Text blocks, formatting and parsing · Example 18

Use formatted and printf for formatting; parseInt parses a numeric String and may throw NumberFormatException. Text blocks need modern Java (15+).

```
/**
 * CHAPTER 2 | Text blocks, formatting and parsing
 * WHY IT MATTERS: Use formatted and printf for formatting; parseInt parses a numeric
```

```
 * String and may throw NumberFormatException. Text blocks need modern Java (15+).
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex18StringFormatting {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String price = "125";
        int value = Integer.parseInt(price);
        System.out.printf("Price: $%,.2f%n", (double) value);
        // STEP 2: Compare the output with your prediction.
        System.out.println("Value=" + value);
        String formatted = "Name: %s | Score: %d".formatted("Maya", 95);
        System.out.println(formatted);
        String message = """
                Java supports
                multiline text blocks.
                """;
        System.out.println(message);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Price: $125.00
Value=125
Name: Maya | Score: 95
Java supports
multiline text blocks.
```

5. Operators and expression evaluation

Operators produce results from operands. Understand type promotion, short-circuiting and precedence instead of relying on memorized rules. Parentheses make intent obvious.

Arithmetic, remainder and negatives · Example 19

% is remainder, not mathematical modulo. Division and remainder with negative operands can surprise learners.

```
/**
 * CHAPTER 2 | Arithmetic, remainder and negatives
 * WHY IT MATTERS: % is remainder, not mathematical modulo. Division and remainder with
 * negative operands can surprise learners.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex19Arithmetic {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int x = 11, y = 4;
        // STEP 2: Compare the output with your prediction.
        System.out.println(x + y);       // 15
        System.out.println(x - y);       // 7
        System.out.println(x * y);       // 44
        System.out.println(x / y);       // 2 (integer division)
        System.out.println(x % y);       // 3
        System.out.println(-11 % 4);     // -3
        System.out.println(Math.floorMod(-11, 4)); // 1
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
15
7
44
2
3
-3
1
```

Prefix/postfix increment and compound assignment · Example 20

++x increments before producing the value; x++ produces the previous value before incrementing. Avoid combining many side effects in one expression.

```java
/**
 * CHAPTER 2 | Prefix/postfix increment and compound assignment
 * WHY IT MATTERS: ++x increments before producing the value; x++ produces the previous
 * value before incrementing. Avoid combining many side effects in one expression.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex20Increment {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int x = 5;
        int before = ++x;               // x = 6, before = 6
        int after = x++;                // after = 6, then x = 7
        // STEP 2: Compare the output with your prediction.
        System.out.println(before + " " + after + " " + x);
        byte small = 10;
        small += 1;                     // implicit cast within compound assignment
        // small = small + 1;           // compile error: int to byte
        System.out.println(small);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
6 6 7
11
```

Logical operators and short circuit · Example 21

&& and || skip the right side when the left determines the result. & and | on booleans evaluate both sides.

```java
/**
 * CHAPTER 2 | Logical operators and short circuit
 * WHY IT MATTERS: && and || skip the right side when the left determines the result. &
 * and | on booleans evaluate both sides.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex21Logical {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String name = null;
        boolean safe = name != null && name.length() > 0;
        // STEP 2: Compare the output with your prediction.
        System.out.println(safe);
        int number = 8;
        System.out.println(number > 0 && number < 10);
        System.out.println(number < 0 || number == 8);
        System.out.println(!(number == 8));
        System.out.println(true ^ false); // boolean exclusive OR
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
false
true
true
false
true
```

Precedence and associativity · Example 22

Multiplication binds more tightly than addition; && binds more tightly than ||. Prefer parentheses in nontrivial expressions.

```
/**
 * CHAPTER 2 | Precedence and associativity
 * WHY IT MATTERS: Multiplication binds more tightly than addition; && binds more
 * tightly than ||. Prefer parentheses in nontrivial expressions.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex22Precedence {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        // STEP 2: Compare the output with your prediction.
        System.out.println(2 + 3 * 4);         // 14
        System.out.println((2 + 3) * 4);       // 20
        boolean result = true || false && false;
        System.out.println(result);            // true
        System.out.println((true || false) && false); // false
        int x = 1;
        int y = 2;
        x = y = 9;                             // assignment right-associative
        System.out.println(x + " " + y);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
14
20
true
false
9 9
```

Conditional ?: operator · Example 23

Use ternary for compact value selection, not side-effect-heavy branching. Type rules can affect the result.

```
/**
 * CHAPTER 2 | Conditional ?: operator
 * WHY IT MATTERS: Use ternary for compact value selection, not side-effect-heavy
 * branching. Type rules can affect the result.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex23Ternary {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int age = 17;
        String category = age >= 18 ? "Adult" : "Minor";
        // STEP 2: Compare the output with your prediction.
        System.out.println(category);
        int score = 86;
        String result = score >= 50 ? "Pass" : "Fail";
        System.out.println(result);
        int a = 3, b = 8;
```

```
        int bigger = a > b ? a : b;
        System.out.println(bigger);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Minor
Pass
8
```

Bitwise shifts, masks and XOR · Example 24

Bitwise operations work on binary integer representations. >> sign-extends; >>> shifts in zeros. Shift distances are masked (int: low 5 bits; long: low 6).

```
/**
 * CHAPTER 2 | Bitwise shifts, masks and XOR
 * WHY IT MATTERS: Bitwise operations work on binary integer representations. >> sign-
 * extends; >>> shifts in zeros. Shift distances are masked (int: low 5 bits; long: low
 * 6).
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex24Bitwise {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int flags = 0b0101;
        int mask = 0b0001;
        // STEP 2: Compare the output with your prediction.
        System.out.println((flags & mask) != 0); // test bit
        System.out.println(Integer.toBinaryString(flags | 0b0010));
        System.out.println(Integer.toBinaryString(flags ^ 0b0001));
        System.out.println(8 << 1);      // 16
        System.out.println(-8 >> 1);     // -4, sign-preserving
        System.out.println(-8 >>> 1);    // large positive number
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
true
111
100
16
-4
2147483644
```

6. Decisions: if, else and switch

Conditions must evaluate to boolean (Java does not treat 1 or 0 as true/false). Branches are tested in order; only the first matching if/else-if branch executes.

if / else-if / else and boundaries · Example 25

Order matters. Check the most selective condition before general ones, and test exact boundary values.

```
/**
 * CHAPTER 2 | if / else-if / else and boundaries
 * WHY IT MATTERS: Order matters. Check the most selective condition before general
 * ones, and test exact boundary values.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
```

```
 */
public class Ex25IfElse {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int score = 85;
        if (score >= 90) {
            // STEP 2: Compare the output with your prediction.
        System.out.println("A");
        } else if (score >= 80) {
            System.out.println("B");
        } else if (score >= 70) {
            System.out.println("C");
        } else {
            System.out.println("Needs work");
        }
        // if (1) { }   // compile error: condition must be boolean
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
B
```

Nested conditions and guard clauses · Example 26

Early guards can keep decision logic readable. The dangling else belongs to the nearest unmatched if.

```
/**
 * CHAPTER 2 | Nested conditions and guard clauses
 * WHY IT MATTERS: Early guards can keep decision logic readable. The dangling else
 * belongs to the nearest unmatched if.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex26NestedIf {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int age = 25;
        boolean hasId = false;
        if (age < 18) {
            // STEP 2: Compare the output with your prediction.
        System.out.println("Too young");
        } else if (!hasId) {
            System.out.println("ID required");
        } else {
            System.out.println("Allowed");
        }
        // Braces eliminate ambiguity around nested if/else.
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
ID required
```

Traditional switch and fall-through · Example 27

Traditional switch supports multiple case labels. Without break, execution falls through to the next case.

```
/**
 * CHAPTER 2 | Traditional switch and fall-through
 * WHY IT MATTERS: Traditional switch supports multiple case labels. Without break,
 * execution falls through to the next case.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex27SwitchStatement {
```

```
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int day = 6;
        switch (day) {
            case 1: case 2: case 3: case 4: case 5:
                // STEP 2: Compare the output with your prediction.
        System.out.println("Weekday");
                break;
            case 6: case 7:
                System.out.println("Weekend");
                break;
            default:
                System.out.println("Invalid day");
        }
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Weekend
```

Modern switch expression with arrows · Example 28

Arrow cases do not fall through. Switch expressions yield a value and must cover all possible inputs (typically with default).

```
/**
 * CHAPTER 2 | Modern switch expression with arrows
 * WHY IT MATTERS: Arrow cases do not fall through. Switch expressions yield a value
 * and must cover all possible inputs (typically with default).
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex28SwitchExpression {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String season = "WINTER";
        String label = switch (season) {
            case "WINTER" -> "Cold";
            case "SUMMER" -> "Hot";
            case "SPRING", "FALL" -> "Mild";
            default -> "Unknown";
        };
        // STEP 2: Compare the output with your prediction.
        System.out.println(label);
        int number = 3;
        String kind = switch (number) {
            case 0 -> "zero";
            default -> { yield number > 0 ? "positive" : "negative"; }
        };
        System.out.println(kind);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Cold
positive
```

7. Loops: repetition and control

A loop repeats work while a condition holds. Choose for when iterations are naturally counted, while when repetitions are condition-driven, and do-while when the body must run at least once.

Counted for loop and off-by-one · Example 29

for has initialization, condition and update. The common pattern i < limit runs limit times starting from 0.

```
/**
 * CHAPTER 2 | Counted for loop and off-by-one
 * WHY IT MATTERS: for has initialization, condition and update. The common pattern i <
 * limit runs limit times starting from 0.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex29For {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        for (int i = 0; i < 5; i++) {
            System.out.print(i + " ");   // 0 1 2 3 4
        }
        // STEP 2: Compare the output with your prediction.
        System.out.println();
        for (int i = 5; i >= 1; i--) {
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
0 1 2 3 4
5 4 3 2 1
```

while loop with explicit progress · Example 30

The condition is evaluated BEFORE each iteration. Forgetting to update the variable can cause an infinite loop.

```
/**
 * CHAPTER 2 | while loop with explicit progress
 * WHY IT MATTERS: The condition is evaluated BEFORE each iteration. Forgetting to
 * update the variable can cause an infinite loop.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex30While {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int count = 3;
        while (count > 0) {
            // STEP 2: Compare the output with your prediction.
        System.out.println("Countdown: " + count);
            count--;                    // termination progress
        }
        System.out.println("Go!");
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Countdown: 3
Countdown: 2
Countdown: 1
Go!
```

do-while executes at least once · Example 31

The condition is tested AFTER the body, so the body executes once even if the condition is initially false.

```java
/**
 * CHAPTER 2 | do-while executes at least once
 * WHY IT MATTERS: The condition is tested AFTER the body, so the body executes once
 * even if the condition is initially false.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex31DoWhile {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int count = 0;
        do {
            // STEP 2: Compare the output with your prediction.
        System.out.println("Runs at least once: " + count);
            count++;
        } while (count < 0);
        System.out.println("Done");
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Runs at least once: 0
Done
```

break exits; continue skips to next iteration · Example 32

break ends the nearest loop; continue proceeds to the next iteration. Use sparingly to keep control flow understandable.

```java
/**
 * CHAPTER 2 | break exits; continue skips to next iteration
 * WHY IT MATTERS: break ends the nearest loop; continue proceeds to the next
 * iteration. Use sparingly to keep control flow understandable.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex32BreakContinue {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        for (int n = 1; n <= 10; n++) {
            if (n == 3) continue;      // skip 3
            if (n == 8) break;         // stop before 8
            System.out.print(n + " ");
        }
        // STEP 2: Compare the output with your prediction.
        System.out.println();
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
1 2 4 5 6 7
```

Nested loops and loop variables · Example 33

For each outer iteration, the inner loop runs completely. Total work grows multiplicatively.

```java
/**
 * CHAPTER 2 | Nested loops and loop variables
 * WHY IT MATTERS: For each outer iteration, the inner loop runs completely. Total work
 * grows multiplicatively.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex33NestedLoops {
```

```
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 4; col++) {
                System.out.print(row * col + "\t");
            }
            // STEP 2: Compare the output with your prediction.
        System.out.println();
        }
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
1□2□3□4□
2□4□6□8□
3□6□9□12□
```

Labeled break and continue · Example 34

Labels can escape nested loops, but usually a refactoring is clearer. Label names are not variables.

```
/**
 * CHAPTER 2 | Labeled break and continue
 * WHY IT MATTERS: Labels can escape nested loops, but usually a refactoring is
 * clearer. Label names are not variables.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex34LabeledBreak {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        outer:
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 3; col++) {
                if (row == 2 && col == 2) break outer;
                // STEP 2: Compare the output with your prediction.
        System.out.println("row=" + row + ", col=" + col);
            }
        }
        System.out.println("Exited both loops");
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
row=1, col=1
row=1, col=2
row=1, col=3
row=2, col=1
Exited both loops
```

Three common loop traps · Example 35

Watch boundary conditions, integer arithmetic, and values that fail to make progress. Use long if the counter could overflow.

```
/**
 * CHAPTER 2 | Three common loop traps
 * WHY IT MATTERS: Watch boundary conditions, integer arithmetic, and values that fail
 * to make progress. Use long if the counter could overflow.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex35LoopPitfalls {
    public static void main(String[] args) {
```

```
        // STEP 1: Follow each statement in order; watch how values and types change.
        int sum = 0;
        for (int i = 1; i <= 5; i++) sum += i;
        // STEP 2: Compare the output with your prediction.
        System.out.println("1..5 sum: " + sum); // 15
        int i = 0;
        while (i < 3) {
            System.out.println("step=" + i);
            i++;                        // without increment, infinite loop
        }
        // for (int k = 0; k <= 5; k++) has 6 iterations, not 5.
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
1..5 sum: 15
step=0
step=1
step=2
```

8. Synthesis: predicting output and fixing bugs

Reading unfamiliar code is a core refresher skill. Trace values in order, track expression types, and identify whether the issue occurs at compile time or runtime.

Trace mixed expressions · Example 36

Predict the printed value before running: follow prefix/postfix order and integer division.

```
/**
 * CHAPTER 2 | Trace mixed expressions
 * WHY IT MATTERS: Predict the printed value before running: follow prefix/postfix
 * order and integer division.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex36TraceExpressions {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int a = 4;
        int b = a++ + 2;             // b = 6, a = 5
        int c = ++a * 2;            // a = 6, c = 12
        double d = a / 4;           // int division: 6 / 4 = 1
        // STEP 2: Compare the output with your prediction.
        System.out.println("a=" + a + ", b=" + b + ", c=" + c + ", d=" + d);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
a=6, b=6, c=12, d=1.0
```

Corrected examples of frequent mistakes · Example 37

Compare correct expressions to common errors in comments. Syntax, equality, null, and division are frequent sources of bugs.

```
/**
 * CHAPTER 2 | Corrected examples of frequent mistakes
 * WHY IT MATTERS: Compare correct expressions to common errors in comments. Syntax,
 * equality, null, and division are frequent sources of bugs.
 * QUICK PRACTICE: Predict printed values before running.
```

```
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex37FindTheBug {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String name = "Java";
        if ("Java".equals(name)) System.out.println("Matches");
        // if (name == "Java") ... compares references instead of content
        int total = 5;
        int count = 2;
        double average = (double) total / count;
        // double wrong = total / count; // 2.0, integer division first
        // STEP 2: Compare the output with your prediction.
        System.out.println(average);
        String optional = null;
        if (optional != null && !optional.isBlank()) {
            System.out.println(optional);
        }
        // if (optional.isBlank()) throws NullPointerException
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Matches
2.5
```

Exact decimal arithmetic with BigDecimal · Example 38

For money, construct BigDecimal from decimal String, not a floating-point double; this avoids importing a pre-existing binary approximation.

```
/**
 * CHAPTER 2 | Exact decimal arithmetic with BigDecimal
 * WHY IT MATTERS: For money, construct BigDecimal from decimal String, not a floating-
 * point double; this avoids importing a pre-existing binary approximation.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex38BigDecimalMoney {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        java.math.BigDecimal price = new java.math.BigDecimal("0.10");
        java.math.BigDecimal tax = new java.math.BigDecimal("0.20");
        // STEP 2: Compare the output with your prediction.
        System.out.println(price.add(tax));  // exactly 0.30
        System.out.println(0.1 + 0.2);      // binary float approximation
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
0.30
0.30000000000000004
```

9. Mastery checks and reference

Use these as a short closed-book review before moving to the next Java chapter.

> Predict output Q1 What does `double x = 7 / 2;` store? Q2 What is `-7 % 3`? Q3 Does `"abc" == new String("abc")` return true? Q4 What does `for (int i=0;i<=3;i++)` run: 3 or 4 iterations? Q5 Why is `String s=null; s.length()` unsafe?

Answers A1 3.0, because integer division happens first. A2 -1. A3 false (use equals for contents). A4 four. A5 null is not an object; method invocation raises NullPointerException.

Compile-error challenge Explain why each fails: `byte b=128;` (out of range); `float f=1.5;` (double literal); `var x;` (no initializer); `if(1)` (not boolean); `int z=4.7;` (lossy conversion); `int a=1; int a=2;` in the same scope (duplicate declaration).

Operator precedence pocket reference High to low (simplified): postfix x++; unary ++x, !, ~, cast; multiplicative * / %; additive + -; shifts << >> >>>; relational < <= > >= instanceof; equality == !=; bitwise & ^ |; logical && ||; ternary ?:; assignment = += .... Use parentheses when in doubt.

Java version and next steps All examples target Java 17+ (also compatible with modern Java LTS versions). The runner methods are boilerplate needed to run files independently; classes, methods, arrays, reference types beyond String, and deeper OOP are intentionally deferred. For each file in IntelliJ: open project, set JDK 17+, right-click the file or green run arrow, and run its main method.

Simple Constructs Big Power
Master these fundamentals!

Java Language Refresher – Chapter 2

Variables • Primitive Types • Operators • Conditions • Loops

Key Building Blocks for Spring Boot and Advanced Java

1 Variables & Scope

Store values and control visibility



- Declaration, initialization, assignment
- Meaningful names (lowerCamelCase)
- final and var keywords
- Scope: local, block, method
- Variables must be assigned before use

```
int score = 10;   // declaration + init
int bonus = 3;
score = score + bonus;  // 13
final double PI = 3.14;
var name = "Java";
```

Scope Examples

- Method scope (visible inside method)
- Block scope (visible inside { } only)
- Cannot use variable outside its scope

Common Mistakes

- Using variable before initialization
- Accessing variable outside its scope

2 Primitive Types

8 built-in types with fixed ranges

Type	Size	Range (Approximate)
byte	8 bits	-128 to 127
short	16 bits	-32,768 to 32,767
int	32 bits	-2.1 billion to 2.1 billion
long	64 bits	-9.2e18 to 9.2e18
float	32 bits	~6-7 decimal digits
double	64 bits	~15-16 decimal digits
char	16 bits	0 to 65,535 (Unicode)
boolean	varies	true or false

3 Operators

Perform operations on values

Category	Operators	Example	Result
Arithmetic	+ - * / %	7 % 3	1
Assignment	= += -= *= /=	x *= 5	x = x + 5
Comparison	== != > < >= <=	5 == 5	true
Logical	&& | !	true && false	false
Bitwise	& | ^ ~ << >>	5 << 1	10
Unary	+ - ++ -- !	++x	x = x + 1
Ternary	? :	age > 18 ? "Adult" : "Minor"	

Quick Examples

```
int a = 10, b = 3;
System.out.println(a + b);   // 13
System.out.println(a % b);   // 1
System.out.println(a > b);   // true
```

Operator Precedence (High → Low)

() → unary → * / % → + - → << >> → < <= > >= → == != → & → ^ → | → && → || → ? : → = += -= ...

4 Type Casting & Conversion

Change a value from one type to another

Automatic Widening (safe)

double
float
long
int
short
byte

Explicit Narrowing (cast needed)

Examples

```
int i = 100;
long l = i;          // widening (auto)
double d = i;        // widening (auto)

int n = (int) 9.7;   // narrowing (cast)
byte b = (byte) 130; // 130 → -126 (overflow)
```

Important Pionts

- Widening is automatic and safe.
- Narrowing needs an explicit cast.
- int arithmetic promotes to int.
- Be careful with overflow when narrowing.

5 Strings

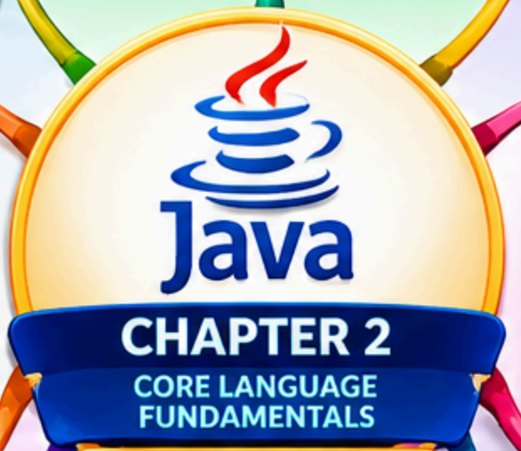
Work with text (not a primitive)

```
String s = "Java";
int len = s.length();              // 4
char ch = s.charAt(1);             // 'a'
String upper = s.toUpperCase();    // "JAVA"
String sub = s.substring(1, 3);    // "av"
boolean eq = s.equals("Java");     // true
```

Common String Operations

- length() – number of characters
- charAt(i) – character at index
- substring(start, end) – part of string
- equals() – compare content
- + – concatenation
- toUpperCase(), toLowerCase()

String is immutable
Methods create new strings. Original string does not change.



6 Conditions & Switch

Make decisions in your code

if – else

```
int age = 18;
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

Ternary Operator

```
String type = (age >= 18) ?
              "Adult" : "Minor";
```

switch (Java 17)

```
switch (day) {
    case 1 -> System.out.println("Mon");
    case 2,3,4,5 -> System.out.println("Weekday");
    default -> System.out.println("Weekend");
}
```

When to use?

- if – simple conditions
- else if – multiple conditions
- switch – many fixed values
- ternary – short expression

Common Mistakes

- Missing break in switch (pre Java 14)
- Using = instead of ==
- Incorrect condition logic

7 Loops

Repeat a block of code

for loop

```
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}
```

Output:

```
1
2
3
```

while loop

```
int i = 1;
while (i <= 3) {
    System.out.println(i);
    i++;
}
```

Output:

```
1
2
3
```

do-while loop

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 3);
```

Output:

```
1
2
3
```

Loop Controls

break	- exit loop
continue	- skip to next iteration
nested loops	- loop inside loop

Common Mistakes

- Infinite loop (condition never false)
- Off-by-one errors
- Forgetting to update counter
- Misusing break/continue

8 Quick Examples & Output

See it all together

```
int age = 46;
double price = 19.95;
boolean isMember = true;
char grade = 'A';
System.out.println(age + " | " + price + " | " + isMember + " | " + grade);
System.out.println("Next age: " + (age + 1));
```

Output:

```
46 | 19.95 | true | A
Next age: 47
```

Key Takeaways

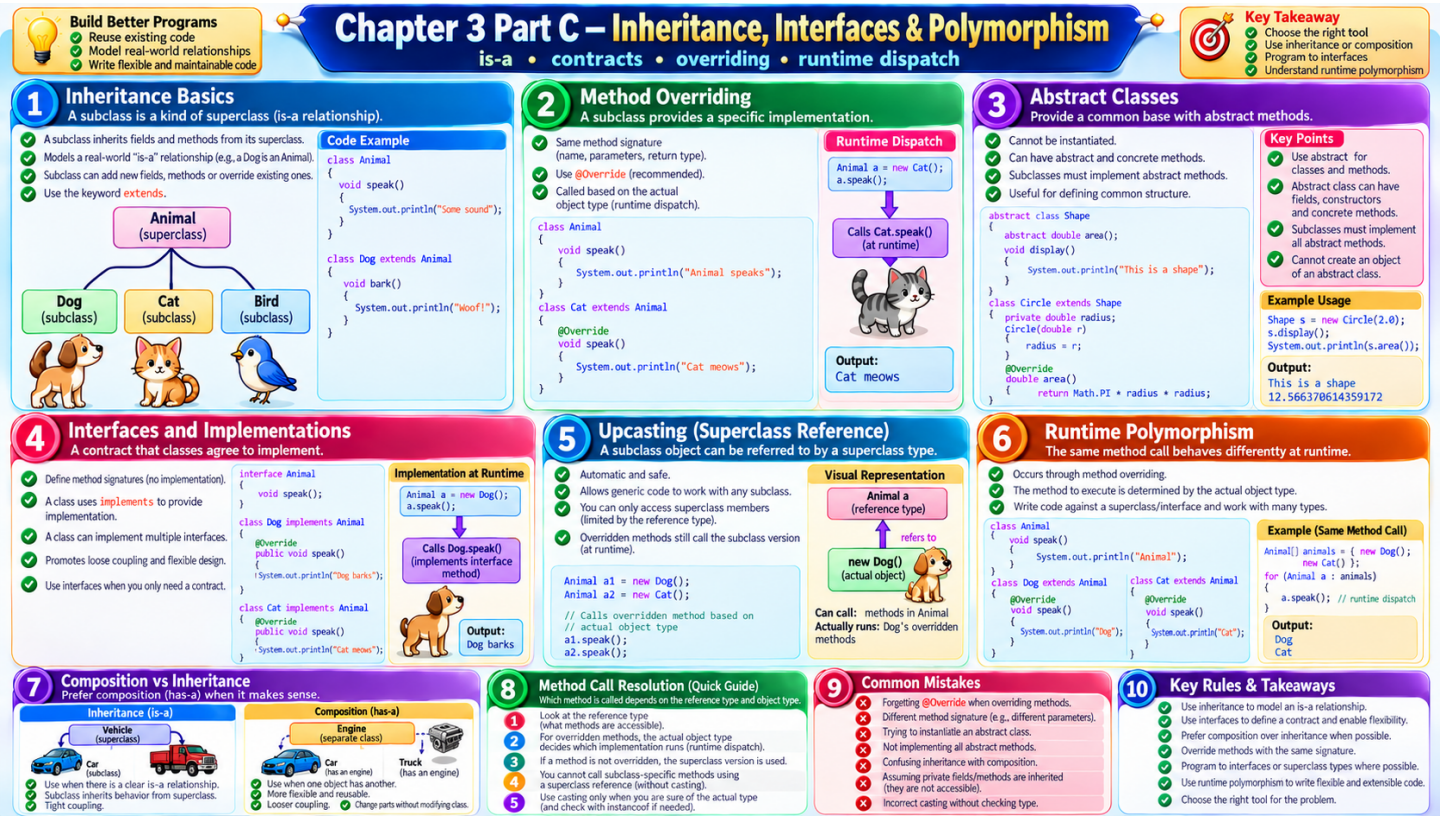
- Choose the right type for the right value.
- Understand operator precedence.
- Use meaningful variable names.
- Write clear and readable conditions and loops.
- Avoid common mistakes.

Chapter 3 — Focused Rebuild

Inheritance, Abstract Classes, Interfaces and Practical Polymorphism

This focused edition rebuilds only the part of Chapter 3 that matters for **inheritance**, **abstract classes** and **interfaces**. The goal is not just to define the terms, but to make the ideas easier to read, easier to compare, and easier to connect to complete runnable code.

You asked for three specific improvements, and this chapter follows them directly: **(1)** start each lesson with a plain-English concept introduction, **(2)** show complete code rather than tiny disconnected fragments, and **(3)** explain the code with notes, examples and relationships so you can see how the ideas fit together in a real Java project.



How to use the infographic: treat it as the quick-glance map, then use the lessons that follow to slow down and understand why each block matters.

1. The big picture first

Why does Java even need these three ideas? Because real systems have many objects that are similar but not identical. A payroll system may have full-time employees, contractors and vendors. A notification system may have email, SMS and Slack senders. A report system may have many report sections with the same skeleton but different content.

If you copy and paste the same fields and methods into every class, the code becomes repetitive and fragile. Java gives you three related tools to control this problem:

- **Inheritance** lets one class reuse and extend another class.
- **Abstract classes** let you define a partially complete base class.
- **Interfaces** let you define a contract that different classes agree to implement.

A practical mental model is this: **inheritance models a family tree**, **abstract classes provide a shared base with some unfinished parts**, and **interfaces describe capabilities**.

2. Inheritance — what it is and why it exists

Inheritance means a subclass can reuse fields and methods from a superclass. You write this using the **extends** keyword. The main reason to use inheritance is to model a clear **is-a** relationship. A savings account is an account. A circle is a shape. A salaried employee is a worker.

Inheritance is most useful when the child really is a specialized form of the parent and the parent contains logic that truly belongs to all children.

- The subclass automatically gets accessible members from the parent.
- The subclass may add new state or behavior.
- The subclass may override inherited methods to provide specialized behavior.
- The parent and child together form a hierarchy.

Complete code — InheritanceBasicsDemo.java

```
package com.refresher.ch3.lesson1;

class Account
{
    private final String accountNumber;
    protected double balance;

    Account(String accountNumber, double balance)
    {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public double getBalance()
    {
        return balance;
    }

    public void deposit(double amount)
    {
        balance += amount;
    }
}
```

```
class SavingsAccount extends Account
{
    private final double annualInterestRate;

    SavingsAccount(String accountNumber, double balance, double annualInterestRate)
    {
        super(accountNumber, balance);
        this.annualInterestRate = annualInterestRate;
    }

    public void applyMonthlyInterest()
    {
        double monthlyRate = annualInterestRate / 12;
        deposit(getBalance() * monthlyRate);
    }

    public void printSummary()
    {
        System.out.println("Savings Account: " + getAccountNumber());
        System.out.println("Balance before interest: 1000.0");
        System.out.println("Balance after interest : " + getBalance());
    }
}

public class InheritanceBasicsDemo
{
    public static void main(String[] args)
    {
        SavingsAccount savings = new SavingsAccount("SB-101", 1000.0, 0.06);
        savings.applyMonthlyInterest();
        savings.printSummary();
    }
}
```

What to notice in this code: Account contains state and behavior that any account would reasonably have. **SavingsAccount** extends it because a savings account is a more specific kind of account. The child class adds one extra concept — interest rate — and one extra behavior — applying interest.

Program output — InheritanceBasicsDemo

```
Savings Account: SB-101
Balance before interest: 1000.0
Balance after interest : 1005.0
```

3. Abstract classes — when a parent should be incomplete on purpose

Sometimes a base class should provide common structure, but it should *not* decide every detail. That is where an abstract class helps. An abstract class can contain fields, constructors, concrete methods and abstract methods. The abstract methods are unfinished on purpose; subclasses must provide them.

Use an abstract class when you want to say, “all of these classes share common identity and some shared behavior, but each child must provide its own implementation for a few core operations.”

- You cannot create an object directly from an abstract class.
- Abstract classes are excellent for shared state plus shared logic.
- A concrete subclass must implement all inherited abstract methods unless it is also abstract.

Complete code — AbstractClassDemo.java

```
package com.refresher.ch3.lesson2;

abstract class ReportSection
{
    private final String title;
```

```
    protected ReportSection(String title)
    {
        this.title = title;
    }

    public String getTitle()
    {
        return title;
    }

    public void printHeader()
    {
        System.out.println("=== " + title + " ===");
    }

    public abstract void printBody();
}

class SalesSummarySection extends ReportSection
{
    private final double totalSales;

    SalesSummarySection(double totalSales)
    {
        super("Sales Summary");
        this.totalSales = totalSales;
    }

    @Override
    public void printBody()
    {
        System.out.println("Total sales this month: " + totalSales);
    }
}

public class AbstractClassDemo
{
    public static void main(String[] args)
    {
        ReportSection section = new SalesSummarySection(42500.75);
        section.printHeader();
        section.printBody();
    }
}
```

What to notice here: the class **ReportSection** already knows how to print the title header. That logic should not be repeated in every subclass. But the base class cannot know the body content for every kind of section. So the body is declared abstract and the subclass **SalesSummarySection** supplies it.

Program output — AbstractClassDemo

```
=== Sales Summary ===
Total sales this month: 42500.75
```

4. Interfaces — contracts and capabilities

An interface answers a different question: not “what family does this class belong to?” but “what capability does this class promise to provide?” A class can implement an interface even if it does not belong to the same inheritance tree as another class.

Interfaces are powerful because they reduce coupling. Your code depends on the contract, not on one concrete implementation. That makes replacement, testing and extension easier.

- Interfaces define a contract.

- A class uses **implements** to agree to that contract.
- A class can implement multiple interfaces.
- Modern interfaces may also contain default and static methods.

Complete code — InterfaceDemo.java

```
package com.refresher.ch3.lesson3;

interface Notifier
{
    void send(String message);
}

class EmailNotifier implements Notifier
{
    @Override
    public void send(String message)
    {
        System.out.println("EMAIL -> " + message);
    }
}

class SmsNotifier implements Notifier
{
    @Override
    public void send(String message)
    {
        System.out.println("SMS   -> " + message);
    }
}

public class InterfaceDemo
{
    public static void main(String[] args)
    {
        Notifier[] notifiers = {
            new EmailNotifier(),
            new SmsNotifier()
        };

        for (Notifier notifier : notifiers)
        {
            notifier.send("Deployment completed successfully");
        }
    }
}
```

What to notice here: the variable type is **Notifier**, while the actual objects are **EmailNotifier** and **SmsNotifier**. That means the caller only cares that each object knows how to **send**. The caller does not need separate logic for every implementation.

Program output — InterfaceDemo

```
EMAIL -> Deployment completed successfully
SMS   -> Deployment completed successfully
```

5. Bringing them together in one realistic example

Now let us put the ideas together in one coherent example. In the payroll example below, we will use:

- **Worker** as an abstract class because every worker has a common identity and shared payroll reporting logic.
- **Payable** as an interface because different objects can promise to calculate pay and tax.
- **Reportable** as an interface because an object can promise that it knows how to build a report line.
- **SalariedEmployee** and **HourlyContractor** as concrete subclasses that each provide their own pay calculation.

Complete code — Payable.java

```
package com.refresher.ch3.payroll.contracts;

import java.math.BigDecimal;

public interface Payable
{
    BigDecimal calculateMonthlyPay();
    BigDecimal calculateTax();
}
```

Complete code — Reportable.java

```
package com.refresher.ch3.payroll.contracts;

public interface Reportable
{
    String buildReportLine();

    default String reportPrefix()
    {
        return "PAYROLL";
    }
}
```

Complete code — Worker.java

```
package com.refresher.ch3.payroll.model;

import java.math.BigDecimal;
import java.math.RoundingMode;

import com.refresher.ch3.payroll.contracts.Payable;
import com.refresher.ch3.payroll.contracts.Reportable;

public abstract class Worker implements Payable, Reportable
{
    private final int workerId;
    private final String name;

    protected Worker(int workerId, String name)
    {
        this.workerId = workerId;
        this.name = name;
    }

    public int getWorkerId()
    {
        return workerId;
    }

    public String getName()
    {
        return name;
    }
```

```
    public BigDecimal netPay()
    {
        return calculateMonthlyPay().subtract(calculateTax()).setScale(2, RoundingMode.HALF_UP);
    }

    @Override
    public String buildReportLine()
    {
        return String.format(
            "%s | id=%d | name=%s | gross=%s | tax=%s | net=%s",
            reportPrefix(),
            workerId,
            name,
            calculateMonthlyPay(),
            calculateTax(),
            netPay()
        );
    }

    public abstract String workerType();
}
```

Complete code — SalariedEmployee.java

```
package com.refresher.ch3.payroll.model;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class SalariedEmployee extends Worker
{
    private final BigDecimal annualSalary;
    private final BigDecimal monthlyAllowance;

    public SalariedEmployee(int workerId, String name, BigDecimal annualSalary, BigDecimal monthlyAllowance
    {
        super(workerId, name);
        this.annualSalary = annualSalary;
        this.monthlyAllowance = monthlyAllowance;
    }

    @Override
    public BigDecimal calculateMonthlyPay()
    {
        return annualSalary.divide(BigDecimal.valueOf(12), 2, RoundingMode.HALF_UP)
            .add(monthlyAllowance)
            .setScale(2, RoundingMode.HALF_UP);
    }

    @Override
    public BigDecimal calculateTax()
    {
        return calculateMonthlyPay().multiply(new BigDecimal("0.20")).setScale(2, RoundingMode.HALF_UP);
    }

    @Override
    public String workerType()
    {
        return "SalariedEmployee";
    }
}
```

Complete code — HourlyContractor.java

```
package com.refresher.ch3.payroll.model;
```

```
import java.math.BigDecimal;
import java.math.RoundingMode;

public class HourlyContractor extends Worker
{
    private final BigDecimal hourlyRate;
    private final int monthlyHours;

    public HourlyContractor(int workerId, String name, BigDecimal hourlyRate, int monthlyHours)
    {
        super(workerId, name);
        this.hourlyRate = hourlyRate;
        this.monthlyHours = monthlyHours;
    }

    @Override
    public BigDecimal calculateMonthlyPay()
    {
        return hourlyRate.multiply(BigDecimal.valueOf(monthlyHours)).setScale(2, RoundingMode.HALF_UP);
    }

    @Override
    public BigDecimal calculateTax()
    {
        return calculateMonthlyPay().multiply(new BigDecimal("0.10")).setScale(2, RoundingMode.HALF_UP);
    }

    @Override
    public String workerType()
    {
        return "HourlyContractor";
    }
}
```

Complete code — PayrollApp.java

```
package com.refresher.ch3.payroll.app;

import java.math.BigDecimal;
import java.util.List;

import com.refresher.ch3.payroll.model.HourlyContractor;
import com.refresher.ch3.payroll.model.SalariedEmployee;
import com.refresher.ch3.payroll.model.Worker;

public class PayrollApp
{
    public static void main(String[] args)
    {
        List<Worker> workers = List.of(
            new SalariedEmployee(101, "Riya", new BigDecimal("120000"), new BigDecimal("750")),
            new HourlyContractor(205, "Arjun", new BigDecimal("80"), 140)
        );

        for (Worker worker : workers)
        {
            System.out.println("-----------------------------------------------");
            System.out.println("Type       : " + worker.workerType());
            System.out.println("Report line: " + worker.buildReportLine());
        }
    }
}
```

How the design works:

- **Worker** is abstract because Java should never create a generic “worker” whose pay rules are unknown.
- **Worker** contains common state such as id and name, and common logic such as **netPay()** and **buildReportLine()**.
- **SalariedEmployee** and **HourlyContractor** inherit from **Worker** and override the calculation methods.
- The loop in **PayrollApp** stores different subclasses in a single **List<Worker>**. This is practical polymorphism: one loop, many behaviors.

Program output — PayrollApp

```
--------------------------------------------
Type       : SalariedEmployee
Report line: PAYROLL | id=101 | name=Riya | gross=10750.00 | tax=2150.00 | net=8600.00
--------------------------------------------
Type       : HourlyContractor
Report line: PAYROLL | id=205 | name=Arjun | gross=11200.00 | tax=1120.00 | net=10080.00
```

6. Decision guide — inheritance vs abstract class vs interface

Use when...	Best tool	Why
You have a clear is-a relationship and reusable base behavior	Inheritance	A child class can reuse and extend parent logic
You need shared state + common methods + some unfinished methods	Abstract class	A partially complete base class fits best
You only need a capability / contract	Interface	Keeps code flexible and loosely coupled
You need multiple capabilities on one class	Interface	Java allows multiple interfaces but only one class inheritance path

7. Common mistakes to avoid

- Using inheritance just to reuse code even when there is no real is-a relationship.
- Forgetting @**Override** on overridden methods.
- Trying to instantiate an abstract class.
- Confusing “contract” with “shared state”: interfaces define behavior promises, while abstract classes can carry state.
- Writing logic against a concrete class when an interface or superclass would be more flexible.

A good rule is: **prefer interfaces for contracts, abstract classes for shared partial implementation, and inheritance only when the hierarchy is conceptually sound.**

8. Quick recap

- Inheritance = reuse + specialization through **extends**.
- Abstract class = a base class with both completed and unfinished parts.
- Interface = a contract that different classes can implement.
- Polymorphism = one reference type can work with many actual object types.
- Good OO design is not about using all these tools everywhere; it is about choosing the right one for the problem.

If you want, the next step after this focused chapter can be to rebuild the remaining parts in the same concept-first style: clearer introductions, full code, and a single realistic example that grows step by step.

Chapter 4
Essential Java Types for Business Applications

WHY THIS CHAPTER MATTERS

Most production bugs involving values are not syntax bugs. They come from comparing the wrong things, unexpected nulls, binary floating-point rounding, or accidentally mixing a local date-time with an absolute instant.

What you will be able to do

- Choose the right type for text, numbers, statuses, money and time.
- Explain equality, null, immutability and formatting rules.
- Use BigDecimal without introducing silent decimal errors.
- Model calendar dates, wall-clock times and UTC instants correctly.

HOW TO STUDY

Read the explanation; trace the code before looking at output; test the variation in IntelliJ. Code panels are focused teaching snippets; the companion project contains complete compilable examples.

23 practical lessons • illustrated reference poster • executable companion project

CHAPTER 4 • ESSENTIAL JAVA TYPES

Text • wrappers • enums • precision • dates • utility APIs

WHY THIS MATTERS

Core Java building blocks for reliable business applications.

MASTER THESE

- String is immutable; compare content with equals().
- Use BigDecimal(String) for decimal money.
- java.time distinguishes dates, instants and zones.

KEY TAKEAWAY

CHOOSE A TYPE THAT MATCHES YOUR DATA

1 String & Equality

Immutable text and comparison

- String stores text; operations create new String values.
- Use equals() for content; == checks object identity.
- length() counts UTF-16 code units, not always characters.

QUICK MODEL

Same content
Different objects
Use equals()

JAVA EXAMPLE • illustrative snippet

```
String a = "Java";
String b = new String("Java");
System.out.println(a.equals(b));
System.out.println(a == b);
```

OUTPUT true | false

2 StringBuilder

Efficient mutable text

- StringBuilder changes its own character sequence.
- Prefer it for repeated concatenation in a loop.
- StringBuffer is synchronized; rarely needed for new code.

QUICK MODEL

String
immutable
Builder: mutable

JAVA EXAMPLE • illustrative snippet

```
StringBuilder sb = new StringBuilder("Hi");
sb.append(" Java");
System.out.println(sb);
```

OUTPUT Hi Java

3 Wrapper Types

Autoboxing, null and identity

- Integer, Double and Boolean wrap primitives.
- Boxing/unboxing is automatic in many expressions.
- Unboxing a null wrapper throws NullPointerException.

QUICK MODEL

Integer ↔ int
Nullable wrapper
Avoid == for boxed values

JAVA EXAMPLE • illustrative snippet

```
Integer n = Integer.valueOf(42);
int value = n;
System.out.println(value);
```

OUTPUT 42

4 Enums

Named, type-safe values

- Enums model a fixed set of legitimate values.
- They can declare fields, constructors and methods.
- Enum constants are compared safely with ==.

QUICK MODEL

Status.NEW
Status.PAID
Type-safe

JAVA EXAMPLE • illustrative snippet

```
enum Status
{
    NEW, PAID
}
System.out.println(Status.PAID.name());
```

OUTPUT PAID

5 BigDecimal

Money and exact decimal rules

- Construct decimal values from strings, not double.
- Use add/subtract/multiply and controlled division.
- equals checks scale too; compareTo compares numeric value.

QUICK MODEL

0.1 + 0.2
double ≠ exact
BigDecimal exact

JAVA EXAMPLE • illustrative snippet

```
BigDecimal a = new BigDecimal("0.10");
BigDecimal b = new BigDecimal("0.20");
System.out.println(a.add(b));
```

OUTPUT 0.30

6 java.time

Date, time and time zones

- LocalDate is a calendar date with no time or zone.
- Instant represents a UTC timeline point.
- ZonedDateTime accounts for zone rules and DST.

QUICK MODEL

LocalDate: date
Instant: point
Zone: local rules

JAVA EXAMPLE • illustrative snippet

```
LocalDate day = LocalDate.of(2026, 10, 11);
System.out.println(day.plusDays(7));
```

OUTPUT 2026-10-18

WATCH OUT • COMMON MISTAKES

- Using == to compare String content
- Using double for decimal money
- Unboxing null wrappers
- Assuming LocalDate has a timezone

QUICK RECALL • BEST PRACTICES

Utility helpers: Objects.requireNonNull, Arrays.toString, Math, UUID.

Chapter 4 reminder: immutable value types simplify business logic.

CHAPTER 4 — ESSENTIAL JAVA TYPES: STRINGS, WRAPPERS, ENUMS & BIGDECIMAL

Read Better • Compare Safely • Model Business Values Correctly

WHY IT MATTERS

- These types appear constantly in business code.
- Strings hold user input, IDs, messages and file data.
- Wrappers represent optional numeric/boolean values.
- Enums model fixed sets such as statuses and types.
- BigDecimal handles money and precise calculations.

REAL-WORLD EXAMPLES

- User input and text data (String)
- Prices, amounts and rates (BigDecimal)
- Optional IDs, flags and counts (Wrappers)
- Statuses, roles and types (Enums)

KEY TAKEAWAYS

- Choose the right type for your data and business rules.
- Strings are immutable; use equals() for comparison.
- Wrappers can be null; be careful with auto-unboxing.
- Enums provide type-safe named constants.
- Use BigDecimal for money; compare with compareTo().
- These types make your code safer, clearer and more maintainable.

2 STRING ESSENTIALS

Immutable text and common operations

- String is immutable; once created, it cannot be changed.
- Use string literals ("...") or new String().
- Use equals() to compare content, not ==.
- Common methods help with everyday text processing.

Common String Methods

Method	What it does
length()	Returns the number of characters.
isBlank()	Checks if empty or only whitespace.
substring(a, b)	Extracts a portion (start inclusive, end exclusive).
contains(s)	Checks if text contains a substring.
startsWith(s)	Checks if text starts with a prefix.
split(regex)	Splits into an array using a delimiter.
replace(old, new)	Replaces matching text.
String.format()	Creates formatted text with placeholders.

JAVA EXAMPLE – String operations

```java
String text = "  Hello Java World  ";
System.out.println(text.trim());
System.out.println(text.length());
System.out.println(text.contains("Java"));
String[] parts = text.trim().split(" ");
System.out.println(String.fromat("First: %s", parts[0]));
```

OUTPUT

```
Hello Java World
19
true
First: Hello
```

3 STRING PITFALLS

Avoid common problems with text

Null safety
A String can be null. Call methods only on non-null values or use Optional/"".

Trimming
Input may contain leading/trailing spaces. Use trim() or isBlank().

Case sensitivity
equals() is case-sensitive. Use equalsIgnoreCase() when appropriate.

Performance
Avoid repeated concatenation in loops. Use StringBuilder for many appends.

JAVA EXAMPLE – Avoid repeated concatenation

```java
String result = "";
for (int i = 0; i < 3; i++) {
    result += "" + i;       // creates many objects
}
System.out.println(result.trim());

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 3; i++) {
    sb.append(" ").append(i);    // efficient
}
System.out.println(sb.toString().trim());
```

OUTPUT

```
0 1 2
0 1 2
```

4 WRAPPER TYPES

Objects for primitive values

- Wrapper classes: Integer, Long, Double, Boolean, Character.
- Support autoboxing and unboxing.
- Can be null (unlike primitives).
- Use valueOf() or parse methods to convert text.
- Integer caches small values (−128 to 127).

Common Wrapper Types

Type	Primitive	parse / valueOf	Notes
Integer	int	parseInt() valueOf()	Caches −128..127
Long	long	parseLong()	Use for large values
Double	double	parseDouble()	For decimal numbers
Boolean	boolean	parseBoolean() valueOf()	true/false text
Character	char	valueOf()	Represents a single character

JAVA EXAMPLE – Autoboxing, parsing and null pitfall

```java
Integer a = 10;             // autoboxing
int b = a;                  // unboxing
Integer c = Integer.valueOf("42");
Double d = Double.parseDouble("3.14");
Boolean flag = Boolean.valueOf("true");

Integer n = null;
// int x = n;    // throws NullPointerException
if (n != null) {
    int x = n;   // safe after null check
    System.out.println(x);
}
```

5 ENUMS

Named constants for a fixed set of values

- Enums define a fixed set of named constants.
- Use in switch statements.
- Enums can have fields, constructors and methods.
- Safer and clearer than using strings for statuses.

JAVA EXAMPLE – Enum with fields and switch

```java
enum OrderStatus {
  NEW("New order"), PAID("Paid"), SHIPPED("Shipped");

  private final String label;
  OrderStatus(String label) { this.label = label; }
  public String getLabel() { return label; }
}

OrderStatus status = OrderStatus.PAID;
System.out.println(status.getLabel());

switch (status) {
  case NEW     -> System.out.println("Process");
  case PAID    -> System.out.println("Ship");
  case SHIPPED -> System.out.println("Done");
}
```

OUTPUT

```
Paid
Ship
```

6 BIGDECIMAL

Exact decimal arithmetic for money

- Represents decimal numbers exactly (as strings), avoiding floating-point errors.
- Use BigDecimal(String) for exact values.
- Scale controls decimal places; use rounding for division.
- compareTo() compares numeric value; equals() also considers scale.
- Use add, subtract, multiply, divide with RoundingMode.

JAVA EXAMPLE – Money calculation

```java
import java.math.BigDecimal;
import java.math.RoundingMode;

BigDecimal prics = new BigDecimal("19.99");
BigDecimal qty = new BigDecimal("3");
BigDecimal total = price.multiply(qty);   // 59.97
BigDecimal tax = total.multiply(new BigDecimal("0.10"))
                .setScale(2, RoundingMode.HALF_UP);
BigDecimal grand = total.add(tax);

System.out.println("Total: " + total);
System.out.println("Tax: " + tax);
System.out.println("Grand: " + grand);
```

OUTPUT

```
Total: 59.97
Tax: 5.997
Grand: 65.967
```

7 COMMON MISTAKES

These lead to bugs and incorrect results

Using double for money
double cannot represent decimals exactly. Use BigDecimal.

Comparing strings with ==
== compares references, not content. Use equals().

NullPointerException from auto-unboxing
Unboxing a null wrapper to a primitive throws NPE. Always check for null.

Using equals() on BigDecimal
equals() also compares scale precisely for numeric comparison.

8 QUICK RULES / AT A GLANCE

Remember these key pointe

1. Use equals() to compare string content (not ==).
2. Check for null before unboxing wrapper types.
3. Use valueOf() or parseXxx() to convert from text.
4. Prefer enums over strings for statuses and types.
5. Use BigDecimal for money and precise calculations.
6. Use compareTo() (not equals()) for BigDecimal numeric comparison.
7. Use StringBuilder for many concatenations in loops.
8. Choose the smallest appropriate numeric type (Integer, Long, etc.).

CHAPTER 4 — ESSENTIAL JAVA TYPES: DATES, TIMES, FORMATTING & PARSING

Use Time Correctly • Parse Carefully • Respect Time Zones

JAVA REFRESHER

WHY THIS MATTERS

- Dates and times are tricky: time zones, daylight saving time (DST), different formats and business rules.
- Mistakes cause real production issues: wrong timestamps, missed deadlines, incorrect reports and failed integrations.
- The java.time API (Java 8+) provides a modern, immutable, clear and powerful model to handle time correctly.

MENTAL MODEL: THREE WAYS TO REPRESENT TIME

Local Values (no time zone)
`LocalDate` `LocalTime` `LocalDateTime`
Human calendar values. Not an instant. No time zone.

→

Absolute Timestamp (on the global timeline)
`Instant`
A single point on the timeline (UTC). Use for storage, events and comparisons.

→

Time-Zone Aware (an instant + zone)
`ZonedDateTime`
An instant with a zone that understands regional rules (e.g. DST). Use for display and zone-based logic.

COMMON MISTAKES

- Using LocalDateTime for an absolute instant (no zone).
- Wrong pattern letters (MM vs mm, yyyy vs YYYY).
- Ignoring time zones and DST.
- Assuming system default zone is correct.
- Mutability assumptions (all java.time types are immutable).
- Parsing user input without validation.
- Storing local date/time without zone when an instant is required.

1 CORE TYPES AND WHEN TO USE THEM

Type	What it represents	When to use	Example
LocalDate	Date (no time, no zone)	Birthdays, due dates, calendar dates	`LocalDate.of(2026, 10, 18)`
LocalTime	Time (no date, no zone)	Opening hours, daily times	`LocalTime.of(14, 30)`
LocalDateTime	Date and time (no zone)	Local app logic, user input (without zone)	`LocalDateTime.of( 2026, 10, 18, 14, 30)`
Instant	Point on timeline (UTC)	Storing, logging, events, comparison across zones	`Instant.now()`
ZonedDateTime	Date, time and zone	Display to users, zone rules, conversions	`ZonedDateTime.now( ZoneId.of("Europe/London"))`
Duration	Time-based amount (seconds, aanos)	Elapsed time between instants	`Duration.ofHours(2)`
Period	Date-based amount (years, months, days)	Calendar calculations (e.g. age, due date)	`Period.ofDays(10)`

2 PARSING AND FORMATTING

- Use DateTimeFormatter to parse and format date/time values.
- Java provides built-in ISO formatters and custom patterns.
- Parsing can fail — handle `DateTimeParseException`.
- Formatting is locale-sensitive when using localized patterns.

ISO FORMATS (BUILT-IN)

- `LocalDate.parse("2026-10-18")`
- `LocalDateTime.parse( "2026-10-18T14:30:00")`
- `Instant.parse("2026-10-18T12:30:00Z")`

COMMON PATTERN LETTERS

`yyyy`	year (e.g. 2026)
`MM`	month (01–12)
`dd`	day of month (01–31)
`HH`	hour (00–23)
`mm`	minute (00–59)
`ss`	second (00–59)

JAVA EXAMPLE • Formatting and parsing

```
DateTimeFormatter f = DateTimeFormatter.ofPattern("dd MMM yyyy HH:mm");
LocalDateTime dt = LocalDateTime.parse("18 Oct 2026 14:30", f);
String out = dt.format(f);
System.out.println(out);
```

OUTPUT 18 Oct 2026 14:30

3 TIME ZONES AND CONVERSIONS

- ZoneId identifies a time zone (e.g. UTC, Europe/London).
- ZonedDateTime understands DST and regional rules.
- Convert between zones, or between local and instant.
- Store Instant in databases/systems; display in user's zone.

JAVA EXAMPLE • Zone conversions and DST

```
ZoneId london = ZoneId.of("Europe/London");
LocalDateTime local = LocalDateTime.of(2026, 3, 29, 1, 30);
ZonedDateTime zdt = local.atZone(london);
ZonedDateTime ny = zdt.withZoneSameInstant(ZoneId.of("America/New_York"));
Instant instant = zdt.toInstant();
System.out.println("London: " + zdt);
System.out.println("New York: " + ny);
System.out.println("Instant: " + instant);
```

OUTPUT

```
London:   2026-03-29T01:30+00:00[Europe/London]
New York: 2026-03-28T21:30-04:00[America/New_York]
Instant:  2026-03-29T01:30:00Z
```

DST NOTE: Some local times may not exist or may be ambiguous
On 2026-03-29 in Europe/London, clocks move forward. 01:30 becomes 02:30.
Use ZonedDateTime and zone rules to handle such cases.

4 DATE ARITHMETIC AND DIFFERENCES

- Use plusXxx/minusXxx for calendar arithmetic.
- Use Period for date-based amounts (years, months, days).
- Use Duration for time-based amounts (seconds, nanos).
- All types are immutable — operations return new instances.

JAVA EXAMPLE • Aritmettic and differences

```
LocalDate date = LocalDate.of(2026, 1, 15);
LocalDate nextMonth = date.plusMonths(1);
Period p = Period.between(date, LocalDate.of(2026, 3, 20));
Instant start = Instant.parse("2026-01-15T10:00:00Z");
Instant end = Instant.parse("2026-01-15T13:30:00Z");
Duration d = Duration.between(start, end);
System.out.println(npeatMonth);
System.out.println(p.getMonths() + " months");
System.out.println(d.toHours() + " hours " + d.toMinutesPart() + " minutes");
```

OUTPUT

```
2026-02-15
2 months
3 hours 30 minutes
```

5 BUSINESS EXAMPLES

Due date (date only) — Use `LocalDate` for deadlines, invoices, subscriptions.

```
LocalDate due = LocalDate.now().plusDays(14);
```

OUTPUT `2026-11-01`

End of day timestamp — Last moment of a day in a specific zone.

```
LocalDate date = LocalDate.of(2026, 10, 18);
ZonedDateTime end = date.atTime(23, 59, 59)
                        .atZone(ZoneId.of("Europe/London"));
```

OUTPUT `2026-10-18 T23:59:59+01:00 [Europe/London]`

Audit timestamp — Use `Instant` for system logs and events.

```
Instant now = Instant.now();
```

OUTPUT `2026-10-18T14:22:30.1232`

User-facing display — Store `Instant`, display in user's zone and format.

```
Instant event = Instant.parse("2026-10-18T14:00:00Z");
ZoneId userZone = ZoneId.of("America/New_York");
DateTimeFormatter f = DateTimeFormatter.ofPattern(
  "dd MMM yyyy, HH:mm z");
String text = event.atZone(userZone).format(f);
```

OUTPUT `18 Oct 2026, 10:00 EDT`

6 AT A GLANCE: QUICK RULES

- Use LocalDate for dates, LocalTime for times.
- Use Instant for storage, events and cross-zone comparison.
- Use ZonedDateTime for user display and zone-based logic.
- Prefer ISO formats for interchange.
- Always handle parsing errors.
- Be explicit about time zones (use ZoneId, not system default).
- Remember: java.time types are immutable.

TYPE COMPARISON

Type	Has Date	Has Time	Has Zone	Represents Instant
LocalDate	✓	✗	✗	✗
LocalTime	✗	✓	✗	✗
LocalDateTime	✓	✓	✗	✗
Instant	✓	✓	(UTC)	✓
ZonedDateTime	✓	✓	✓	✓

7 PARSING FAILURES AND LOCALE

- Parsing throws `DateTimeParseException` on invalid input.
- Validate user input and show clear error messages.
- Use a specific Locale for month/day names (e.g. "Oct").

JAVA EXAMPLE • Handling parse errors

```
try {
    LocalDate.parse("31-02-2026",   DateTimeFormatter.ofPattern("dd-MM-yyyy"));
} catch (DateTimeParseException e) {
        System.out.println("Invalid date: " + e.getMessage());
}
```

OUTPUT Invalid date: Text '31-02-2026' could not be parsed ...

8 QUICK SUMMARY

- Local values = no zone (LocalDate, LocalTime, LocalDateTime).
- Instant = absolute point in time (UTC).
- ZonedDateTime = instant + zone with DST rules.
- Use DateTimeFormatter for parsing/formatting.
- Use Period for date amounts, Duration for time amounts.
- Store Instant, display in user zone.
- Handle parsing errors. Be explict with zones.
- java.time is immutable and thread-safe.

CHAPTER 4 / LESSON 01 OF 23

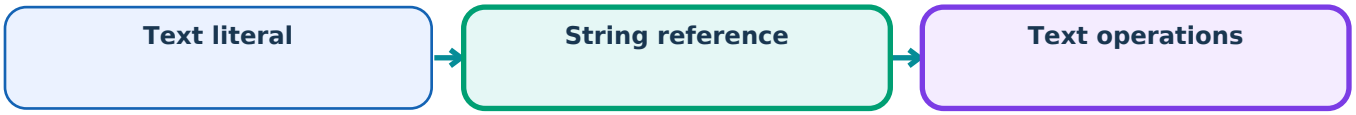
String is a reference type, not a primitive

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String stores text and is a reference type. Its variable can refer to an object or be null; a char stores one UTF-16 code unit.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String city = "Chicago";
char first = city.charAt(0);
System.out.println(city.length());
System.out.println(first);
```

EXPECTED OUTPUT / RESULT

```
7
C
```

EXECUTION TRACE / READ THE STEPS

1. The literal creates or retrieves a String instance and assigns a reference value.
2. length() reports seven UTF-16 code units in this ASCII example.
3. charAt(0) reads the first UTF-16 code unit, C.

DEEPER UNDERSTANDING / WHY IT MATTERS

Unlike primitive numeric variables, a String reference can be null. Calling a method on null throws NullPointerException. String itself is final and immutable: its contents cannot be changed in place. Java automatically provides convenient syntax with double quotes and +, but that convenience does not turn String into a primitive.

REAL-WORLD DECISION / EDGE CASE

Choose String for names, identifiers, messages, and other textual values. Do not use char for a general Unicode character when supplementary characters are possible.

COMMON PITFALL

Treating String as a numeric type for quantities leads to sorting and arithmetic surprises.

PRACTICE / CHANGE ONE THING

What happens after city = null; city.length()?

CHECK YOUR ANSWER

The method call throws NullPointerException.

String content versus reference identity

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use equals() to compare text content. == compares whether two references identify the same object.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);
System.out.println(a.equals(b));
```

EXPECTED OUTPUT / RESULT

```
false
true
```

EXECUTION TRACE / READ THE STEPS

1. Each new String call creates a distinct object.
2. == compares object identity and returns false.
3. equals() checks characters and returns true.

DEEPER UNDERSTANDING / WHY IT MATTERS

String literals may reuse interned objects, which makes == appear to work in some examples. That is an implementation-backed optimization, not the right semantic test for business text. When a value might be null, use a known constant as the receiver: "ACTIVE".equals(status). For nullable variables use Objects.equals(a,b).

REAL-WORLD DECISION / EDGE CASE

equalsIgnoreCase() is useful in limited contexts, but business identifiers may require a clearly defined case-normalization policy.

COMMON PITFALL

Using == for user input or database text may pass tests with literals and fail later.

PRACTICE / CHANGE ONE THING

Will "Java" == "Java" often be true? Should you rely on it?

CHECK YOUR ANSWER

Both literal expressions refer to the interned literal; yes here, but do not use == for text content.

CHAPTER 4 / LESSON 03 OF 23

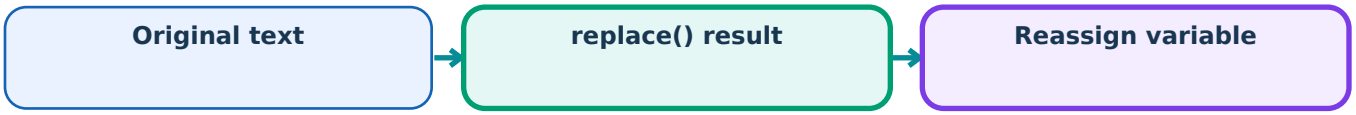
Immutability and reassignment

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String methods never edit the existing String instance; they produce a new String result when a change is needed.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String text = "red";
text.replace("r", "b");
System.out.println(text);
text = text.replace("r", "b");
System.out.println(text);
```

EXPECTED OUTPUT / RESULT

```
red
bed
```

EXECUTION TRACE / READ THE STEPS

1. The first replace() computes a new text value and it is ignored.
2. The original immutable String remains red.
3. The second result is assigned back, so the variable refers to bed.

DEEPER UNDERSTANDING / WHY IT MATTERS

Immutability makes strings safe to share: a helper cannot unexpectedly edit the same String instance that another component holds. The variable itself is not immutable unless the variable is final. Rebinding a variable is different from mutating an object. For repeated string assembly use StringBuilder because repeatedly concatenating in a loop can create many temporary objects.

REAL-WORLD DECISION / EDGE CASE

String is an ideal value-type-like reference: avoid exposing internal mutable arrays when implementing classes that hold textual data.

COMMON PITFALL

Assuming .toUpperCase(), trim(), or replace() mutates the receiver.

PRACTICE / CHANGE ONE THING

What prints after String s=" a "; s.trim(); System.out.print(s.length()); ?

CHECK YOUR ANSWER

3: the result of trim() was discarded.

CHAPTER 4 / LESSON 04 OF 23

Indexing, length and Unicode text

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String.length() counts UTF-16 code units, not necessarily human-visible characters or Unicode code points.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String emoji = "\uD83D\uDE80";
System.out.println(emoji.length());
System.out.println(emoji.codePointCount(0, emoji.length()));
```

EXPECTED OUTPUT / RESULT

```
2
1
```

EXECUTION TRACE / READ THE STEPS

1. The rocket emoji is represented by a surrogate pair.
2. length() returns the two UTF-16 code units.
3. codePointCount() reports one Unicode code point.

DEEPER UNDERSTANDING / WHY IT MATTERS

For ASCII text the three notions often coincide: code units, code points, and perceived characters. For emoji, accented sequences, and many languages they diverge. Even codePointCount does not count user-perceived grapheme clusters such as some emoji sequences. This matters when validating names or trimming text to UI limits. Java charAt() works with code units; codePoints() can be more appropriate for full Unicode code points.

REAL-WORLD DECISION / EDGE CASE

Do not reject all non-ASCII characters just to simplify validation. Use code-point-aware logic when the requirement is about Unicode characters.

COMMON PITFALL

A maximum length of 10 is ambiguous until you define UTF-16 units versus code points versus displayed characters.

PRACTICE / CHANGE ONE THING

What is the length of the single rocket emoji in Java String?

CHECK YOUR ANSWER

2 UTF-16 code units.

EXPANSION 01 / AFTER ORIGINAL LESSON 04

Unicode: characters, code units and code points

WHY THE DISTINCTION MATTERS

Java String uses UTF-16 code units. A visible character is not necessarily one char, and some user-perceived characters consist of multiple Unicode code points.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  String s = "A\uD83D\uDE00";
2  System.out.println(s.length());
3  System.out.println(s.codePointCount(0, s.length()));
4  System.out.println(s.substring(1).codePointAt(0));
```

EXPECTED OUTPUT

```
3
2
128512
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The letter A uses one UTF-16 code unit; the emoji is represented by a surrogate pair, so length() returns 3 rather than 2. codePointCount() returns two Unicode code points. charAt(1) would return only the high surrogate, not the entire emoji. Code-point iteration is safer when an algorithm processes Unicode characters rather than UTF-16 units.

PRODUCTION RULE OR FAILURE MODE

Even code points are not identical to user-perceived grapheme clusters (for example, letters with combining accents or joined emoji). For cursor movement and localized text selection, use character-break algorithms rather than assuming one code point per visual character.

TRY THIS, THEN REVEAL THE REASON

Challenge: If a string contains a letter followed by one emoji, how many results would chars().count() and codePoints().count() return?
Reasoned answer: 3 and 2, respectively: chars() iterates UTF-16 code units, whereas codePoints() recognizes the surrogate pair.

CHAPTER 4 / LESSON 05 OF 23

substring boundaries and off-by-one rules

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

substring(start,end) includes start but excludes end. Index positions start at zero.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String code = "ABCD1234";
System.out.println(code.substring(0, 4));
System.out.println(code.substring(4));
```

EXPECTED OUTPUT / RESULT

```
ABCD
1234
```

EXECUTION TRACE / READ THE STEPS

1. Positions 0 through 3 form the first result.
2. The end index 4 is excluded.
3. substring(4) takes text from position 4 to the end.

DEEPER UNDERSTANDING / WHY IT MATTERS

The exclusive end boundary aligns with array/list slice ideas and makes length equal to end-start. Invalid indexes throw StringIndexOutOfBoundsException. Never assume external IDs have a fixed length unless the contract guarantees it. For variable-length fields prefer delimiter parsing with explicitly checked indexes. Note that substring does not mutate its source.

REAL-WORLD DECISION / EDGE CASE

When extracting data from a URL, filename or account ID, validate boundaries before slicing.

COMMON PITFALL

Treating end as inclusive drops or adds a character in production mapping code.

PRACTICE / CHANGE ONE THING

What does "123456".substring(2,5) produce?

CHECK YOUR ANSWER

345.

Text cleanup: strip, trim and replace

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Prefer strip() to remove Unicode whitespace, and replace() for literal text replacement; know when regex is involved.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String raw = "  Chicago  ";
System.out.println(raw.strip());
System.out.println(raw.replace("Chicago", "IL"));
```

EXPECTED OUTPUT / RESULT

```
Chicago
  IL
```

EXECUTION TRACE / READ THE STEPS

1. strip() returns a new String with surrounding Unicode whitespace removed.
2. replace(CharSequence,CharSequence) replaces literal content.
3. raw remains unchanged for both operations.

DEEPER UNDERSTANDING / WHY IT MATTERS

trim() is older and removes only characters up to U+0020 at both ends; strip() uses Character.isWhitespace and handles more Unicode whitespace, though not every visually blank character. replaceAll() accepts a regular expression, not a literal token. Validation should distinguish normalization from silently changing an identifier. Reject unexpected text rather than over-correcting business keys.

REAL-WORLD DECISION / EDGE CASE

Use a documented normalization policy for inbound names and codes; keep original values when auditing requires exact input.

COMMON PITFALL

replaceAll(".", "") removes every character because dot is a regex wildcard.

PRACTICE / CHANGE ONE THING

What happens when raw.replace("Chicago","IL") is called without assignment?

CHECK YOUR ANSWER

A new value is produced but raw still contains the original string.

Splitting, joining and regex hazards

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String.split() takes a regular expression; Pattern.quote protects literal delimiters that contain regex metacharacters.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String input = "A|B|C";
String[] parts = input.split("\\|");
System.out.println(parts.length);
System.out.println(String.join("-", parts));
```

EXPECTED OUTPUT / RESULT

```
3
A-B-C
```

EXECUTION TRACE / READ THE STEPS

1. A pipe has special meaning in regular expressions.
2. The regex escaped pipe matches the literal | delimiter.
3. join() combines the resulting values using a hyphen.

DEEPER UNDERSTANDING / WHY IT MATTERS

In Java source, a backslash is also escaped, so the regex literal pipe needs two backslashes in the Java source string: "\\|". A simpler general solution is input.split(Pattern.quote(delimiter)). split() normally drops trailing empty fields: "A,".split(",") produces one element. Use split(",", -1) if the trailing empty value is significant.

REAL-WORLD DECISION / EDGE CASE

For CSV with quoted commas, use a CSV parser rather than a naive split.

COMMON PITFALL

Using split(".") does not split on a literal period.

PRACTICE / CHANGE ONE THING

How do you preserve trailing empty fields when splitting A,?

CHECK YOUR ANSWER

Call split(",", -1).

EXPANSION 02 / AFTER ORIGINAL LESSON 07

Text at boundaries: normalize, split and validate

WHY THE DISTINCTION MATTERS

Input often arrives with extra whitespace, arbitrary capitalization, and separators that are regex metacharacters. Treat parsing as a boundary concern rather than scattering string fixes across business logic.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  String raw = "  a|b|c  ";
2  String normalized = raw.strip();
3  String[] parts = normalized.split("\\|");
4  System.out.println(parts.length);
5  System.out.println(parts[1]);
```

EXPECTED OUTPUT

```
3
b
```

UNDER THE HOOD / FOLLOW THE EXECUTION

strip() removes Unicode whitespace according to its definition, whereas trim() only handles characters at or below U+0020. split() takes a regex, not an ordinary literal separator; the vertical bar means alternation in regex and therefore must be escaped. The Java string literal itself also processes backslashes, creating two levels of escaping. In production, Pattern.quote(delimiter) makes a dynamic delimiter safe.

PRODUCTION RULE OR FAILURE MODE

String.split(regex) discards trailing empty fields by default. For a fixed-column import, use split(regex, -1) to retain trailing empty fields and validate the expected field count before converting values.

TRY THIS, THEN REVEAL THE REASON

Challenge: What is the difference between "A,B,".split(",").length and "A,B,".split(",", -1).length?
Reasoned answer: 2 versus 3. The default split drops trailing empty strings; negative limit keeps them.

StringBuilder and efficient assembly

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use StringBuilder for repeated changes to a text buffer; StringBuilder is mutable and not thread-safe.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
StringBuilder builder = new StringBuilder();
builder.append("Order-");
builder.append(42);
System.out.println(builder.toString());
```

EXPECTED OUTPUT / RESULT

```
Order-42
```

EXECUTION TRACE / READ THE STEPS

1. A mutable builder begins empty.
2. append() adds text and the numeric value to the same builder.
3. toString() provides an immutable String snapshot.

DEEPER UNDERSTANDING / WHY IT MATTERS

Ordinary + concatenation is clear and efficient for a few pieces; the Java compiler may optimize it. In a repeated loop or complex assembly, StringBuilder makes the mutation intentional and usually avoids unnecessary intermediate strings. StringBuffer is synchronized and is rarely needed in new ordinary application code. Never share one StringBuilder across threads without coordination.

REAL-WORLD DECISION / EDGE CASE

Do not use StringBuilder as a substitute for parameterized SQL queries or safe HTML escaping.

COMMON PITFALL

Repeated String concat inside a very large loop can allocate many intermediate values.

PRACTICE / CHANGE ONE THING

Does builder.append("!") change a previously returned String snapshot?

CHECK YOUR ANSWER

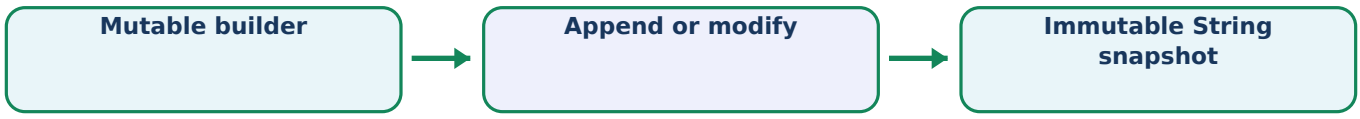
No. The String snapshot remains unchanged.

EXPANSION 03 / AFTER ORIGINAL LESSON 08

StringBuilder: mutation, snapshot and iteration cost

WHY THE DISTINCTION MATTERS

Text concatenation with + is clear for a few values. A builder is useful when assembling a variable amount of text in a loop or producing a structured message incrementally.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  StringBuilder sb = new StringBuilder("A");
2  String snapshot = sb.toString();
3  sb.append("B");
4  System.out.println(snapshot);
5  System.out.println(sb);
6  sb.deleteCharAt(0);
7  System.out.println(sb);
```

EXPECTED OUTPUT

```
A
AB
B
```

UNDER THE HOOD / FOLLOW THE EXECUTION

A StringBuilder is mutable: append, insert, replace and delete alter its contents. The String returned by toString() does not change after the builder is subsequently mutated. Avoid repeated result = result + item within large loops when a builder communicates intent and can avoid repeated copying. The compiler can optimize simple concatenation expressions, so do not replace straightforward + usage everywhere.

PRODUCTION RULE OR FAILURE MODE

StringBuilder is not designed for synchronized shared mutation. Prefer one builder owned by one task; share the immutable String once construction is complete. StringBuffer is synchronized but is rarely the right default for modern code.

TRY THIS, THEN REVEAL THE REASON

Challenge: After saving a String snapshot, can a later append() modify that original String?
Reasoned answer: No. The builder changes, but an already-created String remains immutable.

CHAPTER 4 / LESSON 09 OF 23

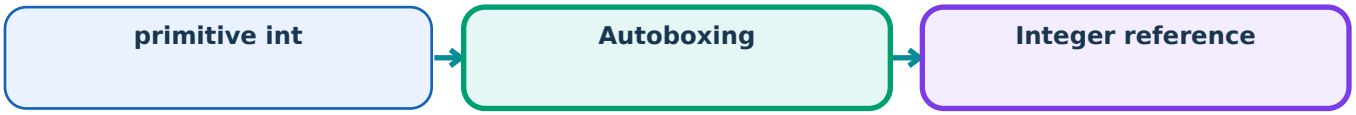
Wrapper types and autoboxing

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Wrapper classes such as Integer and Double allow primitive-like values in generic containers, but they remain references.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
int count = 4;
Integer boxed = count;
int again = boxed;
System.out.println(again + 1);
```

EXPECTED OUTPUT / RESULT

```
5
```

EXECUTION TRACE / READ THE STEPS

1. Java automatically boxes int 4 to an Integer value.
2. Unboxing retrieves the primitive int.
3. Arithmetic uses the primitive and prints 5.

DEEPER UNDERSTANDING / WHY IT MATTERS

Generics operate on reference types, so List<Integer> is allowed and List<int> is not. Boxing and unboxing are convenient but can add allocations, null risk, and overhead in tight numeric loops. BigDecimal is not a wrapper for double; it is a decimal arithmetic type with its own scale and rounding rules.

REAL-WORLD DECISION / EDGE CASE

Choose primitives for required numeric fields with no missing state; wrappers are useful for nullable fields and generic APIs.

COMMON PITFALL

Accidentally treating an Integer as guaranteed non-null.

PRACTICE / CHANGE ONE THING

Can List<int> compile?

CHECK YOUR ANSWER

No; List<Integer> is the correct generic form.

CHAPTER 4 / LESSON 10 OF 23

Null unboxing and safe defaults

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Unboxing a null wrapper throws NullPointerException even when the target variable is primitive.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Integer count = null;
System.out.println(count == null);
int safe = count == null ? 0 : count;
System.out.println(safe);
```

EXPECTED OUTPUT / RESULT

```
true
0
```

EXECUTION TRACE / READ THE STEPS

1. The wrapper variable contains null.
2. The ternary checks null before selecting a branch.
3. Only a non-null wrapper would be unboxed in the second branch.

DEEPER UNDERSTANDING / WHY IT MATTERS

The dangerous pattern is int n = count; with a nullable Integer. A null check on the reference must happen before arithmetic, relational operators, or primitive assignment that triggers unboxing. When zero is not a valid stand-in for missing data, keep the missing state explicit rather than silently defaulting. APIs should make their nullability contract clear.

REAL-WORLD DECISION / EDGE CASE

Prefer database and JSON mapping contracts that distinguish missing, empty, and zero when business meaning differs.

COMMON PITFALL

An innocent-looking count > 0 can throw NPE if count is Integer and null.

PRACTICE / CHANGE ONE THING

What happens with Integer x=null; System.out.println(x+1);?

CHECK YOUR ANSWER

NullPointerException during unboxing.

CHAPTER 4 / LESSON 11 OF 23

Wrapper equality and cached instances

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use equals() for wrapper value equality after handling null; == tests object identity.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Integer a = Integer.valueOf(200);
Integer b = Integer.valueOf(200);
System.out.println(a.equals(b));
System.out.println(a.intValue() == b.intValue());
```

EXPECTED OUTPUT / RESULT

```
true
true
```

EXECUTION TRACE / READ THE STEPS

1. Integer.valueOf creates or returns an Integer with value 200.
2. equals() compares contained integer values.
3. Unboxing compares primitive ints, both 200.

DEEPER UNDERSTANDING / WHY IT MATTERS

Integer caching guarantees reuse for at least -128 through 127; outside that range identity is not guaranteed. Depending on caching and JVM configuration, == between wrapper objects may seem consistent in one environment and differ in another. Comparison operators such as > trigger unboxing, which has null risks. Explicit equals and primitive comparisons express intent better.

REAL-WORLD DECISION / EDGE CASE

For numeric business values, decide whether equality is numeric, textual, or decimal-scaled; they are not always interchangeable.

COMMON PITFALL

Expecting Integer a=100; Integer b=100; a==b behavior to generalize to 1000.

PRACTICE / CHANGE ONE THING

Should two Integer objects with value 200 be compared by ==?

CHECK YOUR ANSWER

No; use equals() or compare their primitive values.

EXPANSION 04 / AFTER ORIGINAL LESSON 11

Wrapper equality, unboxing and null-safe comparisons

WHY THE DISTINCTION MATTERS

Wrapper objects introduce an object identity distinction and a null state that primitives do not have. Small cached integers sometimes make a buggy == comparison appear to work.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Integer a = Integer.valueOf(200);
2  Integer b = Integer.valueOf(200);
3  System.out.println(a.equals(b));
4  System.out.println(a.intValue() == b.intValue());
5  Integer missing = null;
6  System.out.println(java.util.Objects.equals(missing, b));
```

EXPECTED OUTPUT

```
true
true
false
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The two Integer.valueOf(200) calls produce references representing the same numeric value; a portable demonstration compares numeric wrappers by equals() or unboxed values, never by reference identity. The default Integer cache covers -128 through 127, but relying on identity behavior for any value is incorrect. Objects.equals(a,b) handles null references safely. Automatic unboxing to an int can throw NullPointerException when the wrapper contains null.

PRODUCTION RULE OR FAILURE MODE

For nullable input, make the policy explicit: reject null, preserve it, or assign a documented default. Calling map.get(key) + 1 can fail if the map returns a null Integer; getOrDefault also does not replace an explicit null mapping.

TRY THIS, THEN REVEAL THE REASON

Challenge: Why can Integer x = null; int y = x; compile and then fail at runtime?
Reasoned answer: The compiler inserts unboxing. Java attempts to call intValue() on null, throwing NullPointerException.

CHAPTER 4 / LESSON 12 OF 23

Parsing numbers at input boundaries

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Parsing turns text into typed values; invalid input throws exceptions that should be handled at the boundary.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String input = "42";
int amount = Integer.parseInt(input);
System.out.println(amount + 8);
```

EXPECTED OUTPUT / RESULT

```
50
```

EXECUTION TRACE / READ THE STEPS

1. The input is a String containing digits.
2. parseInt validates and converts to int.
3. Normal integer addition produces 50.

DEEPER UNDERSTANDING / WHY IT MATTERS

Integer.parseInt throws NumberFormatException for non-numeric or out-of-range input. You should decide whether leading and trailing whitespace are allowed: trim or strip explicitly if so. Integer.parseInt accepts a leading sign but not decimal points. For monetary values, parse BigDecimal directly from the decimal string to avoid double conversion.

REAL-WORLD DECISION / EDGE CASE

Boundary code should produce useful validation messages rather than exposing raw parsing exceptions to end users.

COMMON PITFALL

Applying + to two strings concatenates; it does not parse and add their values.

PRACTICE / CHANGE ONE THING

What does Integer.parseInt("12.0") do?

CHECK YOUR ANSWER

It throws NumberFormatException.

EXPANSION 05 / AFTER ORIGINAL LESSON 12

Parsing numeric input: validation and overflow

WHY THE DISTINCTION MATTERS

User input is text. Treat invalid format, out-of-range values and null input as separate decisions instead of assuming parseInt will always succeed.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  for (String text : new String[]{"42", "4x"})
2  {
3      try
4      {
5          System.out.println(Integer.parseInt(text));
6      }
7      catch (NumberFormatException ex)
8      {
9          System.out.println("Invalid integer");
10     }
11 }
```

EXPECTED OUTPUT

```
42
Invalid integer
```

UNDER THE HOOD / FOLLOW THE EXECUTION

Integer.parseInt() rejects non-integer characters and values outside the int range with NumberFormatException. Long.parseLong() permits a wider range but is not unlimited. When domain constraints are narrower than the Java type, validate them after parsing. Monetary input should generally be parsed straight into BigDecimal from decimal text, not first converted to double.

PRODUCTION RULE OR FAILURE MODE

Validation belongs at input boundaries. Use a specific error message for the user but retain enough context for diagnosis; avoid logging sensitive raw input indiscriminately.

TRY THIS, THEN REVEAL THE REASON

Challenge: Would Integer.parseInt("2147483648") succeed?
Reasoned answer: No. That is one more than Integer.MAX_VALUE (2,147,483,647), so it throws NumberFormatException.

CHAPTER 4 / LESSON 13 OF 23

Enums model closed sets of states

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

An enum provides named singleton constants for a known set of valid statuses; it is safer than free-form strings.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
enum Status
{
    NEW, ACTIVE, CLOSED
}
Status state = Status.ACTIVE;
System.out.println(state == Status.ACTIVE);
```

EXPECTED OUTPUT / RESULT

```
true
```

EXECUTION TRACE / READ THE STEPS

1. Java declares three enum constants.
2. state refers to the unique ACTIVE enum instance.
3. == is appropriate for enum identity equality.

DEEPER UNDERSTANDING / WHY IT MATTERS

Enums can contain fields, constructors, and methods, and implement interfaces. They can work in switch expressions to keep all known statuses explicit. Persisting enum.name() directly creates a coupling to source names; if changing a name would break external data, map stable external codes instead. Avoid ordinal() as a persisted value because reordering constants changes the number.

REAL-WORLD DECISION / EDGE CASE

Use enums for bounded states such as NEW, RUNNING, FAILED, not for user-extensible lists of categories.

COMMON PITFALL

Persisting ordinal numbers for durable business status values.

PRACTICE / CHANGE ONE THING

Can enum variables be null?

CHECK YOUR ANSWER

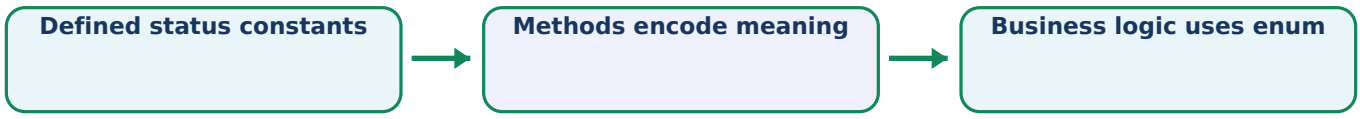
Yes. Enum type does not prevent a reference from being null.

EXPANSION 06 / AFTER ORIGINAL LESSON 13

Enums with fields, methods and controlled transitions

WHY THE DISTINCTION MATTERS

Enums are not merely numbered constants. They are real Java classes with a fixed set of instances, constructors, fields and methods, suitable for validated business statuses.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  enum State
2  {
3      NEW("Open"), SHIPPED("In transit");
4      private final String label;
5      State(String label) { this.label = label; }
6      String label() { return label; }
7  }
8  System.out.println(State.SHIPPED.label());
```

EXPECTED OUTPUT

```
In transit
```

UNDER THE HOOD / FOLLOW THE EXECUTION

An enum constructor is called when its fixed constants are initialized. It cannot be called with new State(...) from outside. Enum fields should usually be final when representing fixed metadata. Compared with free-form strings, enums make invalid statuses harder to introduce and allow exhaustive handling in switch expressions for well-defined cases.

PRODUCTION RULE OR FAILURE MODE

The declaration above is shown as a class-level enum followed by a line that runs in main(). In a real program, separate the enum declaration from the caller. Persisting enum ordinal() is fragile because reordering constants changes numbers; prefer stable string codes for external storage.

TRY THIS, THEN REVEAL THE REASON

Challenge: What happens if you rename an enum constant used as its serialized database value?
Reasoned answer: Existing stored values may no longer deserialize. Use explicit, stable external codes when a persistent contract matters.

CHAPTER 4 / LESSON 14 OF 23

Switch expressions over enums

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Switch expressions return a value and can enforce exhaustive handling of known enum constants.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
enum State
{
    OPEN, CLOSED
}
State s = State.OPEN;
String label = switch (s)
{
    case OPEN -> "Ready";
    case CLOSED -> "Done";
};
System.out.println(label);
```

EXPECTED OUTPUT / RESULT

```
Ready
```

EXECUTION TRACE / READ THE STEPS

1. The selector is the OPEN enum constant.
2. The OPEN arrow arm is selected.
3. The entire switch expression evaluates to Ready.

DEEPER UNDERSTANDING / WHY IT MATTERS

Modern switch expressions use -> to avoid accidental fall-through. A switch expression over all enum constants can be exhaustive without default; adding a new enum constant then forces a compile-time review when the code is recompiled. Avoid a default branch if you want the compiler to remind you about new states; add one when external compatibility requires it.

REAL-WORLD DECISION / EDGE CASE

Switch expressions are especially useful for readable mapping from status to label, priority, or allowed operation.

COMMON PITFALL

Overusing default can hide new enum constants that should receive explicit business handling.

PRACTICE / CHANGE ONE THING

What happens after a new enum constant is added but not handled in an otherwise exhaustive switch? Answer: The switch expression must be updated to compile against the expanded enum.

BigDecimal: decimal money versus floating-point

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use BigDecimal created from a decimal string or valueOf(double) for predictable decimal values; double is binary floating point.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
BigDecimal x = new BigDecimal("0.1");
BigDecimal y = new BigDecimal("0.2");
System.out.println(x.add(y));
```

EXPECTED OUTPUT / RESULT

```
0.3
```

EXECUTION TRACE / READ THE STEPS

1. Each BigDecimal is constructed from an exact decimal string.
2. add() performs arbitrary-precision decimal arithmetic.
3. The result is the exact decimal 0.3.

DEEPER UNDERSTANDING / WHY IT MATTERS

The expression 0.1 + 0.2 using double is not exactly 0.3. new BigDecimal(0.1) captures the exact binary floating-point approximation, producing a long decimal representation. BigDecimal.valueOf(0.1) uses Double.toString and is often suitable when a double has already entered the system, but direct decimal text remains preferable at input boundaries.

REAL-WORLD DECISION / EDGE CASE

Store monetary amounts with an agreed scale and currency. BigDecimal does not automatically know the currency, tax rule, or allowed precision.

COMMON PITFALL

new BigDecimal(0.1) instead of new BigDecimal("0.1").

PRACTICE / CHANGE ONE THING

Should you convert a price string to double before BigDecimal?

CHECK YOUR ANSWER

No. Construct BigDecimal directly from the decimal text.

CHAPTER 4 / LESSON 16 OF 23

BigDecimal scale and rounding contracts

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Division can be non-terminating; choose scale and RoundingMode explicitly when the result requires rounding.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
BigDecimal one = new BigDecimal("1");
BigDecimal three = new BigDecimal("3");
System.out.println(one.divide(three, 2, RoundingMode.HALF_UP));
```

EXPECTED OUTPUT / RESULT

```
0.33
```

EXECUTION TRACE / READ THE STEPS

1. The exact value 1/3 has an infinite decimal expansion.
2. The requested scale is two decimal places.
3. HALF_UP rounds the value to 0.33.

DEEPER UNDERSTANDING / WHY IT MATTERS

divide(divisor) without a rounding context throws ArithmeticException if the quotient cannot be represented exactly as a finite decimal. HALF_UP rounds ties away from zero in magnitude; HALF_EVEN reduces cumulative bias for certain repeated rounding calculations. Round at documented business boundaries rather than after every intermediate operation, otherwise small errors compound. Rounding to two places is not universally correct for every currency or rate.

REAL-WORLD DECISION / EDGE CASE

Make rounding a domain decision, not a hidden default in a utility method.

COMMON PITFALL

Calling one.divide(three) without specifying rounding produces ArithmeticException.

PRACTICE / CHANGE ONE THING

What does new BigDecimal("1.005").setScale(2, HALF_UP) produce?

CHECK YOUR ANSWER

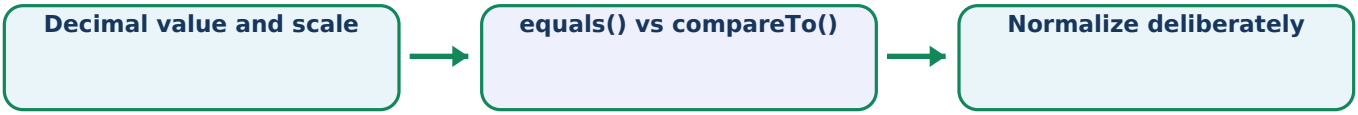
1.01.

EXPANSION 07 / AFTER ORIGINAL LESSON 16

BigDecimal: scale, equality and numeric comparison

WHY THE DISTINCTION MATTERS

Two decimal values can represent the same number but have different scales. That distinction affects equality, HashSet keys, display formatting and financial rules.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  BigDecimal a = new BigDecimal("2.0");
2  BigDecimal b = new BigDecimal("2.00");
3  System.out.println(a.equals(b));
4  System.out.println(a.compareTo(b) == 0);
5  System.out.println(a.scale() + "," + b.scale());
```

EXPECTED OUTPUT

```
false
true
1,2
```

UNDER THE HOOD / FOLLOW THE EXECUTION

equals() considers numeric value and scale, while compareTo() tests numeric ordering and returns zero for equal numerical values. HashSet follows equals/hashCode and therefore may retain 2.0 and 2.00 as separate keys; TreeSet commonly follows compareTo and may consider them duplicates. This can produce surprising differences between hashed and sorted collections.

PRODUCTION RULE OR FAILURE MODE

Decide whether scale is part of the business meaning. Normalize at boundaries when appropriate, but be careful: stripTrailingZeros() can produce a negative scale for values such as 1000, which changes string representation.

TRY THIS, THEN REVEAL THE REASON

Challenge: Should monetary equality checks automatically use equals()?
Reasoned answer: Not always. For numeric equivalence despite scale differences, compareTo()==0 may be the intended rule; domain-specific currency and scale constraints still matter.

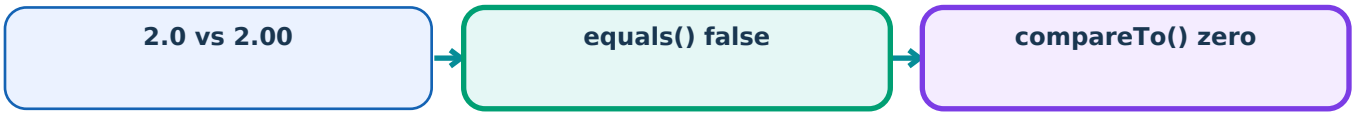
BigDecimal equals, compareTo and collection behavior

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

BigDecimal.equals checks value and scale; compareTo compares numeric value. They may disagree for 2.0 and 2.00.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
BigDecimal a = new BigDecimal("2.0");
BigDecimal b = new BigDecimal("2.00");
System.out.println(a.equals(b));
System.out.println(a.compareTo(b));
```

EXPECTED OUTPUT / RESULT

```
false
0
```

EXECUTION TRACE / READ THE STEPS

1. The first decimal has scale one.
2. The second has scale two.
3. equals() distinguishes scales; compareTo treats the amounts as numerically equal.

DEEPER UNDERSTANDING / WHY IT MATTERS

This difference affects collections: HashSet uses equals()/hashCode(), while TreeSet uses its comparator or natural ordering (compareTo). Therefore a HashSet could retain both values while a naturally ordered TreeSet sees them as equivalent. For monetary comparisons, compareTo()==0 often expresses numeric equality, but choose a consistent scale policy if values are map keys or persisted in equality-sensitive structures.

REAL-WORLD DECISION / EDGE CASE

Do not use BigDecimal as a hash key without understanding whether scale is part of identity.

COMMON PITFALL

Using equals() when the business requirement is numeric equality independent of scale.

PRACTICE / CHANGE ONE THING

Would HashSet and TreeSet necessarily treat 2.0 and 2.00 identically?

CHECK YOUR ANSWER

No; they can differ because equals and compareTo differ.

EXPANSION 08 / AFTER ORIGINAL LESSON 17

Money allocation: rounding once and distributing leftovers

WHY THE DISTINCTION MATTERS

Division of a monetary total among several recipients cannot always be exact at the currency scale. Avoid creating or losing cents through repeated independent rounding.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```java
int totalCents = 100;
int people = 3;
int base = totalCents / people;
int extra = totalCents % people;
for (int i = 0; i < people; i++)
{
    System.out.print((base + (i < extra ? 1 : 0)) + " ");
}
```

EXPECTED OUTPUT

```
34 33 33
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The allocation computes a common integer number of cents per recipient and distributes the remainder one cent at a time. The three shares add back to 100, preserving the invariant that allocated money must equal the original amount. A BigDecimal representation is still appropriate for many financial calculations, but integer minor units can simplify allocation for a known currency scale.

PRODUCTION RULE OR FAILURE MODE

Different currencies use different minor-unit conventions; taxes and interest calculations may require greater precision before rounding. Store the rounding and allocation policies with the business rules, not hidden inside an arithmetic helper.

TRY THIS, THEN REVEAL THE REASON

Challenge: If 101 cents are split among three recipients, what are the allocations using this policy?
Reasoned answer: 34, 34 and 33 cents. The allocations add to 101 without loss.

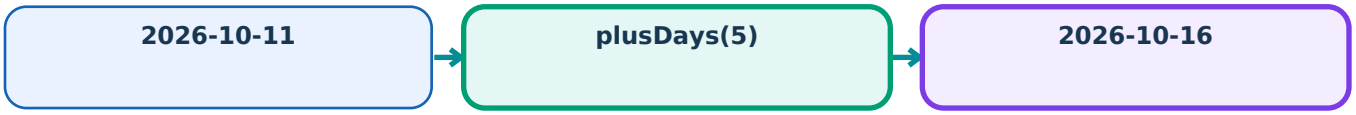
LocalDate for calendar dates

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

LocalDate represents a calendar date with no clock time and no time zone.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
LocalDate date = LocalDate.of(2026, 10, 11);
System.out.println(date.plusDays(5));
```

EXPECTED OUTPUT / RESULT

```
2026-10-16
```

EXECUTION TRACE / READ THE STEPS

1. The date stores year, month, and day.
2. plusDays(5) computes a new calendar date.
3. The original LocalDate remains unchanged.

DEEPER UNDERSTANDING / WHY IT MATTERS

Use LocalDate for birthdays, due dates, and business dates that are calendar-based rather than instants. A LocalDate does not answer what absolute moment midnight represents because no zone is present. java.time classes such as LocalDate are immutable, making them easier to share across layers. Do not represent date-only business information as a midnight Instant unless a specific time zone conversion is intended.

REAL-WORLD DECISION / EDGE CASE

DB DATE and Java LocalDate often correspond, but confirm database drivers and application mapping rules.

COMMON PITFALL

Adding 24 hours to an Instant is not always the same as moving to the next local calendar date across DST.

PRACTICE / CHANGE ONE THING

What does LocalDate.of(2024,2,28).plusDays(1) return?

CHECK YOUR ANSWER

2024-02-29, because 2024 is a leap year.

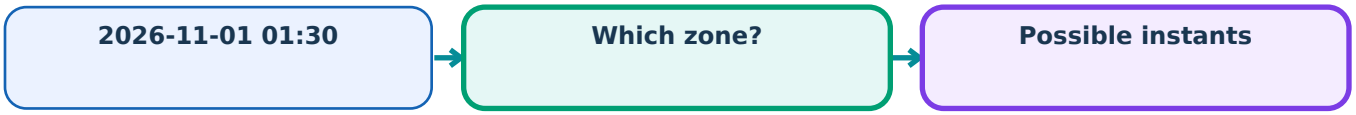
LocalDateTime is not an absolute instant

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

LocalDateTime is a wall-clock date and time without a time zone or offset; it cannot unambiguously identify an instant.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
LocalDateTime slot = LocalDateTime.of(2026, 11, 1, 1, 30);
System.out.println(slot.getHour());
System.out.println(slot.toLocalDate());
```

EXPECTED OUTPUT / RESULT

```
1
2026-11-01
```

EXECUTION TRACE / READ THE STEPS

1. The wall-clock fields are created directly.
2. getHour() reads the hour field.
3. toLocalDate() extracts just the calendar date.

DEEPER UNDERSTANDING / WHY IT MATTERS

This is useful for schedules described by local clocks, but some wall-clock times repeat or disappear during daylight-saving transitions. Converting to Instant requires explicit zone or offset rules. Avoid mixing UTC instants, local date-times, and strings in a domain model without stating what each field means. Storing a LocalDateTime without a zone for an audit event can leave an ambiguous timestamp.

REAL-WORLD DECISION / EDGE CASE

A branch office appointment may need LocalDateTime plus ZoneId; an audit creation time should usually be an Instant.

COMMON PITFALL

Assuming LocalDateTime.now() and Instant.now() are interchangeable values.

PRACTICE / CHANGE ONE THING

Can LocalDateTime identify an unambiguous absolute moment on its own?

CHECK YOUR ANSWER

No.

CHAPTER 4 / LESSON 20 OF 23

Instant and UTC timeline values

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Instant represents an absolute point on the timeline; it is ideal for event timestamps and ordering.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Instant event = Instant.parse("2026-10-11T12:00:00Z");
System.out.println(event);
```

EXPECTED OUTPUT / RESULT

```
2026-10-11T12:00:00Z
```

EXECUTION TRACE / READ THE STEPS

1. The ISO-8601 string includes Z, indicating UTC offset zero.
2. Instant.parse() creates a timeline value.
3. Printing retains the UTC timestamp representation.

DEEPER UNDERSTANDING / WHY IT MATTERS

Different users can display the same Instant in different local zones while referring to one exact event moment. java.time Instant stores epoch-based seconds and nanoseconds. It is not a date-only value, and it should not be formatted as local wall-clock time without an explicit ZoneId. For testable current time, inject a Clock rather than calling now() throughout business logic.

REAL-WORLD DECISION / EDGE CASE

Log creation timestamps and persisted event times are good Instant candidates.

COMMON PITFALL

Treating 2026-10-11T12:00 in Chicago as if it meant 12:00 UTC.

PRACTICE / CHANGE ONE THING

Does changing the viewer’s time zone change the Instant?

CHECK YOUR ANSWER

No. Only its local representation changes.

EXPANSION 09 / AFTER ORIGINAL LESSON 20

LocalDateTime is not an instant: converting explicitly

WHY THE DISTINCTION MATTERS

A local clock value is ambiguous without a time zone. Translating it into a timestamp requires a zone or offset; choosing the wrong zone changes the moment recorded.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  LocalDateTime local = LocalDateTime.of(2026, 1, 1, 12, 0);
2  ZoneId chicago = ZoneId.of("America/Chicago");
3  Instant instant = local.atZone(chicago).toInstant();
4  System.out.println(instant);
```

EXPECTED OUTPUT

```
2026-01-01T18:00:00Z
```

UNDER THE HOOD / FOLLOW THE EXECUTION

Chicago is on standard time in January, UTC-6, so noon locally becomes 18:00 UTC. An Instant uniquely identifies a point on the timeline, while LocalDate and LocalDateTime express human calendar concepts without a zone. Use the type that matches the meaning: birth date as LocalDate, an audit event as Instant, and an appointment rule tied to a location as a local time plus ZoneId.

PRODUCTION RULE OR FAILURE MODE

Never assume server-default time zone is the user or event time zone. Accept an explicit ZoneId and document the behavior for ambiguous or nonexistent local times.

TRY THIS, THEN REVEAL THE REASON

Challenge: Would the exact same LocalDateTime represent the same instant in Tokyo and Chicago?
Reasoned answer: No. The wall-clock components are equal, but the zone offsets differ.

Time zones and daylight-saving behavior

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

ZonedDateTime pairs date-time fields with a zone’s historical and future rules; some local times are skipped or repeated.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
ZoneId zone = ZoneId.of("America/Chicago");
Instant instant = Instant.parse("2026-01-01T12:00:00Z");
System.out.println(instant.atZone(zone).getHour());
```

EXPECTED OUTPUT / RESULT

6

EXECUTION TRACE / READ THE STEPS

1. A January UTC Instant is selected.
2. America/Chicago uses UTC-06:00 on that date.
3. The local hour is 6 AM.

DEEPER UNDERSTANDING / WHY IT MATTERS

Use named region zones such as America/Chicago rather than a hard-coded offset like -06:00 when scheduling across seasons. Offset is a numerical displacement; ZoneId includes daylight-saving and historical rules. During spring transitions some wall-clock times do not exist; during fall transitions some occur twice. APIs resolving overlaps/gaps have defined policies, which must match the application’s business intent.

REAL-WORLD DECISION / EDGE CASE

Financial cutoffs and recurring meetings should use explicit business-zone rules, not the server default zone.

COMMON PITFALL

Converting using a fixed -06:00 offset in July when Chicago is on daylight time (-05:00).

PRACTICE / CHANGE ONE THING

Is ZoneOffset.ofHours(-6) equivalent to America/Chicago year-round?

CHECK YOUR ANSWER

No.

EXPANSION 10 / AFTER ORIGINAL LESSON 21

DST gaps and overlaps: local clocks are not continuous

WHY THE DISTINCTION MATTERS

Some zones skip an hour in spring and repeat an hour in autumn. A local date-time may therefore map to zero, one, or two valid offsets.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  ZoneId zone = ZoneId.of("America/New_York");
2  LocalDateTime gap = LocalDateTime.of(2026, 3, 8, 2, 30);
3  LocalDateTime overlap = LocalDateTime.of(2026, 11, 1, 1, 30);
4  System.out.println(zone.getRules().getValidOffsets(gap).size());
5  System.out.println(zone.getRules().getValidOffsets(overlap).size());
```

EXPECTED OUTPUT

```
0
2
```

UNDER THE HOOD / FOLLOW THE EXECUTION

At the spring-forward transition the 02:30 local time does not exist, so the list of valid offsets is empty. During the fall-back transition, 01:30 occurs twice and has two valid offsets. atZone() applies default resolution rules; when scheduling high-stakes events, the application may need to ask the user to resolve ambiguity or define a consistent business policy.

PRODUCTION RULE OR FAILURE MODE

A calendar recurrence such as “9 a.m. every day” should normally be modeled by LocalTime and ZoneId, not by adding a fixed 24-hour Duration to yesterday’s Instant. Check whether an operation is calendar-based or duration-based.

TRY THIS, THEN REVEAL THE REASON

Challenge: What should happen to a requested appointment at 02:30 during a spring DST gap?
Reasoned answer: There is no universal answer. The application must define whether to reject, move forward, or choose another time.

CHAPTER 4 / LESSON 22 OF 23

Period, Duration and temporal arithmetic

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Period models calendar units (years, months, days); Duration models elapsed time-based units.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Instant start = Instant.parse("2026-01-01T00:00:00Z");
Instant end = start.plusSeconds(90);
System.out.println(Duration.between(start, end).getSeconds());
```

EXPECTED OUTPUT / RESULT

```
90
```

EXECUTION TRACE / READ THE STEPS

1. Two fixed instants identify a 90-second interval.
2. Duration.between calculates timeline elapsed time.
3. getSeconds() prints 90.

DEEPER UNDERSTANDING / WHY IT MATTERS

A period of one day on the local calendar and a duration of 24 hours are not always interchangeable near daylight saving. Period is useful for age, billing-cycle dates, and calendar arithmetic. Duration is useful for timeouts, processing latency and elapsed intervals. LocalDate.plusMonths applies calendar rules, including end-of-month adjustment.

REAL-WORLD DECISION / EDGE CASE

Choose elapsed time for SLA timers and calendar periods for anniversaries or business cycle boundaries.

COMMON PITFALL

Treating plusDays(1) on a zoned date as exactly 24 hours across DST.

PRACTICE / CHANGE ONE THING

Should a timeout of 30 seconds use Period or Duration?

CHECK YOUR ANSWER

Duration.

CHAPTER 4 / LESSON 23 OF 23

DateTimeFormatter, locale and stable output

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Parse and format with explicit patterns and locale/zone when required; do not depend on a machine’s default settings.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
LocalDate d = LocalDate.of(2026, 10, 11);
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("MM/dd/uuuu");
System.out.println(d.format(fmt));
```

EXPECTED OUTPUT / RESULT

```
10/11/2026
```

EXECUTION TRACE / READ THE STEPS

1. The LocalDate is independent of display format.
2. The formatter applies a month/day/year pattern.
3. The result is a String for presentation or interchange.

DEEPER UNDERSTANDING / WHY IT MATTERS

Use predefined ISO formatters for machine-to-machine contracts where possible. In custom patterns, lower-case mm means minutes while upper-case MM means month; yyyy and uuuu also have nuanced era behavior. Formatter instances are immutable and thread-safe. When formatting an Instant as local time, supply a ZoneId; parsing a text without an offset cannot magically recover the original time zone.

REAL-WORLD DECISION / EDGE CASE

Prefer ISO-8601 strings at service boundaries and localized formatting in presentation code.

COMMON PITFALL

Using "mm/dd/yyyy" expecting month but actually printing minutes.

PRACTICE / CHANGE ONE THING

Is DateTimeFormatter thread-safe for reuse?

CHECK YOUR ANSWER

Yes, java.time formatters are immutable and thread-safe.

EXPANSION 11 / AFTER ORIGINAL LESSON 23

Parsing and formatting: machine contracts versus local display

WHY THE DISTINCTION MATTERS

Stable storage and API formats are different from how people expect dates, decimal numbers and labels to appear on-screen.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  LocalDate day = LocalDate.parse("2026-10-11");
2  DateTimeFormatter f = DateTimeFormatter.ofPattern(
3      "dd MMM yyyy", Locale.US);
4  System.out.println(day.format(f));
5  System.out.println(day);
```

EXPECTED OUTPUT

```
11 Oct 2026
2026-10-11
```

UNDER THE HOOD / FOLLOW THE EXECUTION

LocalDate.parse() defaults to an ISO-8601 date. The explicit DateTimeFormatter turns it into a human-friendly representation. Unlike SimpleDateFormat, modern java.time formatters are immutable and safe to share. In pattern syntax, uppercase M means month, lowercase m means minute, and lowercase y denotes year-of-era; pattern letters matter.

PRODUCTION RULE OR FAILURE MODE

Avoid serializing a localized display string as a machine API contract. Prefer ISO-8601 or a documented structured format at service boundaries and apply locale rules at the presentation edge.

TRY THIS, THEN REVEAL THE REASON

Challenge: What would the pattern "dd mm yyyy" accidentally show in a LocalDate formatting operation?
Reasoned answer: It asks for minutes, which a LocalDate does not contain; formatting can fail rather than providing a month.

Chapter 4 — Master Recall Map

A fast scan of the most important invariants and practical decisions. Each section links back to the lessons you just studied.

JAVA CHAPTER 4 | MASTER INFOGRAPHIC

CORE RULES • EXAMPLES • REAL-WORLD DECISIONS

01 STRINGS

- ✓ Immutable text; equals() checks content.
- ✓ Substring end index is exclusive.
- ✓ Unicode code points are not always char values.

02 WRAPPERS

- ✓ Integer/Double are objects, not primitives.
- ✓ Unboxing null throws NullPointerException.
- ✓ Do not rely on wrapper == caching.

03 ENUMS

- ✓ Fixed named set of domain values.
- ✓ Use == on enum constants.
- ✓ Switch on explicit states.

04 MONEY

- ✓ Use new BigDecimal("0.1"), not new BigDecimal(0.1).
- ✓ State rounding mode explicitly.
- ✓ compareTo is numeric; equals includes scale.

05 DATES & TIME

- ✓ LocalDate = date; LocalDateTime = wall-clock.
- ✓ Instant = UTC timeline point.
- ✓ ZonedDateTime = local time plus zone rules.

06 PARSING & FORMATTING

- ✓ Parse strings at the boundary.
- ✓ Specify Locale and ZoneId when needed.
- ✓ Prefer explicit formatters for external data.

Real-world checklist before you move on

REVIEW POINT

☐ Can I state which text equality is intended?

REVIEW POINT

☐ Can a nullable wrapper unbox here?

REVIEW POINT

☐ Is a status best represented as enum or String?

REVIEW POINT

☐ Have I converted monetary text directly to BigDecimal?

REVIEW POINT

☐ Did I specify rounding only where business rules require it?

REVIEW POINT

☐ Does this value represent a date, local time, or absolute Instant?

REVIEW POINT

☐ Have I specified ZoneId and Locale at the proper boundary?

NEXT STEP

Run the matching *Demo.java file in the companion IntelliJ/Maven project. Compare actual console output to EXPECTED_OUTPUT.txt; then change inputs and predict behavior before rerunning.

CHAPTER 5

Collections and Generics — Rebuilt for Understanding

Choose the right data structure, understand its behavior, and learn how type safety and equality work.

How this edition teaches
Every lesson starts with the idea in ordinary English. Then come the key rules, a **complete runnable file**, the actual program output, and notes that connect the code to the concept.

What you will learn

- List vs Set vs Map, ordering, duplicates, and lookup
- ArrayList, HashSet, LinkedHashMap, TreeSet, Queue and iteration
- Generic classes and methods, type invariance, and PECS
- equals()/hashCode() as used by HashSet and HashMap
- Map.merge(), computeIfAbsent(), and safe collection copying
- An integrated inventory program using several collection types together

Chapter 5 – Collections & Generics

List • Set • Map • Iteration • Duplicates • Null Handling • Generics

Powerful Collections
Store, organize and process groups of objects efficiently in modern Java applications!

Key Takeaways
- Choose the right collection type
- Understand ordering and duplicates
- Use generics for type safety
- Write clean, maintainable code

1 Why Collections Matter

Work with groups of objects easily and efficiently.

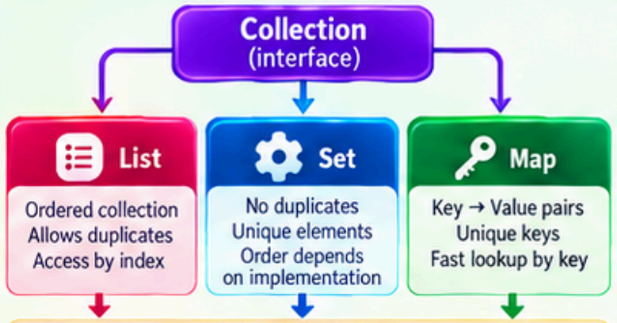
- Store and manage large amounts of data
- Provide useful operations (add, remove, search)
- Reduce boilerplate compared to arrays
- Power modern applications (web, APIs, databases)
- Well-designed framework in java.util
- Use generics for type safety and cleaner code

Real-World Examples

- **E-commerce:** list of products, customer data
- **User management:** store and find users
- **Analytics:** collect and process event data
- **Caching:** fast lookups with keys
- **Reporting:** group, sort and filter results

2 The Collection Families

Different collection types for different needs.



3 Key Characteristics

Quick comparison of List, Set and Map.

Feature	List	Set	Map
Stores	Single values	Single values	Key → Value pairs
Order	Maintains insertion order (ArrayList)	Depends (HashSet: no order, LinkedHashSet: insertion order)	Depends (HashMap: no order, LinkedHashMap: insertion order; TreeMap: sorted)
Duplicates	Allowed	Not allowed	Keys not allowed (values can repeat)
Null values	Allowed (ArrayList)	One null element (HashSet/LinkedHashSet) Not allowed (TreeSet)	One null key (HashMap/LinkedHashMap) Not allowed (TreeMap)
Access	By index (get, set)	No index (use iteration)	By key (get, put)
Best for	Ordered data with duplicates	Unique elements	Mapping keys to values

4 Common Implementations

Popular classes and what they are best for.

Collection	Class	Key Features	Best For
List	ArrayList	Fast random access	General purpose
	LinkedList	Fast add/remove at ends	Frequent insertions/deletions
	Vector	Synchronized (legacy)	Older multi-threaded code
Set	HashSet	No order, fast lookup	Uniqueness, fast ops
	LinkedHashSet	Maintains insertion order	Ordered unique collection
	TreeSet	Sorted order (natural/comparator)	Sorted unique data
Map	HashMap	No order, fast lookup	General purpose
	LinkedHashMap	Maintains insertion order	Predictable iteration order
	TreeMap	Sorted by key (natural/comparator)	Sorted key-value pairs

5 Working with Collections

Create, add, access and iterate over collections.

List Example

Ordered, allows duplicates.

```
List<String> names =
    new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Alice");

for (String name : names)
{
    System.out.println(name);
}
```

Output
```
Alice
Bob
Alice
```

Set Example

No duplicates.

```
Set<String> names =
    new HashSet<>();
names.add("Alice");
names.add("Bob");
names.add("Alice");

for (String name : names)
{
    System.out.println(name);
}
```

Output
```
Alice
Bob
```

Map Example

Key-value pairs.

```
Map<String, Integer> ages =
    new HashMap<>();
ages.put("Alice", 25);
ages.put("Bob", 30);
ages.put("Charlie", 25);

for (Map.Entry<String, Integer>
    entry : ages.entrySet())
{
    System.out.println(entry.getKey()
        + " = " + entry.getValue());
}
```

Output
```
Alice = 25
Bob = 30
Charlie = 25
```

6 Iteration Patterns

Multiple ways to iterate over collections.

For-each Loop (simple)

```
List<String> names =
    List.of("A", "B", "C");
for (String n : names)
{
    System.out.println(n);
}
```

Output
```
A
B
C
```

Iterator (more control)

```
Iterator<String> it =
    names.iterator();
while (it.hasNext())
{
    String n = it.next();
    if (n.equals("B"))
    {
        it.remove(); // safe
    }
}
```

Output
```
[A, C] // after removal
```

Stream (functional style)

```
names.stream()
    .filter(n -> n.startsWith("A"))
    .forEach(System.out::println);
```

Output
```
A
```

Map Iteration (entries)

```
for (Map.Entry<String, Integer>
    e : ages.entrySet())
{
    System.out.println(e.getKey()
        + " -> " + e.getValue());
}
```

Output
```
Alice -> 25
Bob -> 30
Charlie -> 25
```

7 Generics – Type Safe Collections

Use generics to store a specific type and catch errors at compile time.

Why Use Generics?

- Type safety (compile-time checking)
- No need for explicit casts
- Cleaner, more readable code
- Prevents accidental wrong types
- Works with all collection types

Generic Type Syntax

```
Collection<Type> name =
    new Implementation<>();
```

Example: Generic Collections

```
List<String> words = new ArrayList<>();
words.add("Java");
words.add("Collections");
words.add("Generics");

String first = words.get(0);  // no cast needed
// words.add(123);  // compile-time error!
```

Output
```
Java
```

Common Mistake

✗ **Using raw types (no generics).**

```
List items = new ArrayList();
items.add("Java");
items.add(123);  // compiles, but unsafe!
```

8 Important Rules & Best Practices

Avoid common pitfalls and write better code.

1. **Choose the right collection**
   List for ordered duplicates, Set for unique elements, Map for key-value pairs.
2. **Know about duplicates**
   Lists allow duplicates. Sets do not. Maps do not allow duplicate keys.
3. **Understand ordering**
   ArrayList maintains insertion order. HashSet has no order. LinkedHashSet preserves insertion order. TreeSet/TreeMap keep elements sorted.
4. **Handle null values**
   Most collections allow null. TreeSet and TreeMap do not.
5. **Use generics**
   Always specify the type (e.g., List<String>). Avoid raw types.
6. **Pick the right implementation**
   ArrayList for fast access, LinkedList for frequent insert/remove, HashMap for fast lookups, TreeMap for sorted keys.
7. **Use iteration wisely**
   For-each for simple cases, Iterator when you need to remove, Streams for powerful filtering.
8. **Keep it clean**
   Use meaningful names, prefer interfaces (List, Set, Map), and follow best practices for maintainable code.

Chapter 5 – Collections Choices

List • Set • Map • Queue & Deque • Iteration • Choosing the Right Collection

Different collections for different needs
Choose wisely!

1 Why Collections Matter

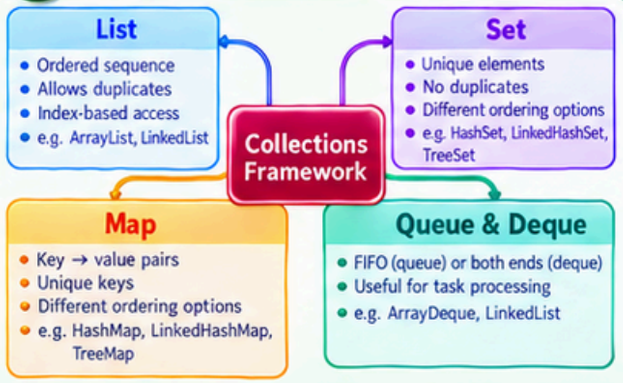
More than just arrays!

Collections store and manage groups of objects with useful operations. Choosing the right collection improves correctness, performance and maintainability.

- Built-in, tested implementations
- Handle common data structures
- Save development time
- Choose based on behavior and requirements

2 The Big Picture

Main collection types and their purpose



3 Choosing the Right Collection

Quick guide for common scenarios

Need	Recommended Collection
Ordered list, fast access, add/remove at end	ArrayList
Frequent insertions/removals in the middle	LinkedList
Unique elements, any order	HashSet
Unique elements, keep insertion order	LinkedHashSet
Unique elements, sorted order	TreeSet
Key → value pairs, any order	HashMap
Key → value pairs, preserve insertion order	LinkedHashMap
Key → value pairs, sorted by key	TreeMap
Process elements in FIFO order	Queue (e.g. ArrayDeque)
Add and remove from both ends	Deque (e.g. ArrayDeque)

4 Iteration Options

Common ways to iterate through collections

Enhanced for-each loop (all collections)

```
List<String> names = Arrays.asList("Alice", "Bob");
for (String name : names)
{
    System.out.println(name);
}
```

Iterator (works for all collections)

```
Iterator<String> it = names.iterator();
while (it.hasNext())
{
    String name = it.next();
    System.out.println(name);
}
```

forEach (Java 8+, all collections)

```
names.forEach(name ->
{    System.out.println(name);
});
```

5 List Implementations

Ordered collection, allows duplicates

Implementation	Key Idea	Typical Use	Main Operations
ArrayList	• Resizable array • Fast random access • Slow inserts/removes in middle	Read-heavy, fast index-based access	add(), get(), set(), remove(), size()
LinkedList	• Doubly linked list • Fast inserts/removes anywhere • Slower random access	Frequent insertions and deletions, especially in middle	add(), get(), remove(), addFirst(), addLast()

Example: List operations

```
List<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.set(0, "Apricot");
System.out.println(list.get(0));    // Apricot
System.out.println(list.size());    // 2
```

Output:

```
Apricot
2
```

6 Set Implementations

Unique elements (no duplicates)

Implementation	Key Idea	Ordering	Typical Use
HashSet	• Hash table • No duplicates • Fast add/contains	No guaranteed order	Fast lookups when order doesn't matter
LinkedHashSet	• Hash table + linked list • No duplicates	Insertion order	Unique elements and keep insertion order
TreeSet	• Balanced tree • No duplicates	Sorted (natural or custom)	Unique elements in sorted order

Example: Set behavior

```
Set<String> set = new LinkedHashSet<>();
set.add("C");
set.add("A");
set.add("B");
set.add("A");   // duplicate ignored
System.out.println(set);
```

Output:

```
[C, A, B]
```

7 Map Implementations

Key → value pairs (unique keys)

Implementation	Key Idea	Ordering	Typical Use
HashMap	• Hash table • Unique keys • Fast put/get	No guaranteed order	General purpose key-value storage
LinkedHashMap	• Hash table + linked list • Unique keys	Insertion order	Remember order of entries
TreeMap	• Balanced tree • Unique keys	Sorted by key (natural or custom)	Keys in sorted order

Example: Map operations

```
Map<String, Integer> map = new LinkedHashMap<>();
map.put("Alice", 30);
map.put("Bob", 25);
map.put("Carol", 27);
System.out.println(map.get("Bob"));
System.out.println(map);
```

Output:

```
25
{Alice=30,
 Bob=25,
 Carol=27}
```

8 Queue and Deque

FIFO processing and both-end access

Implementation	Key Idea	Typical Use
Queue (e.g. ArrayDeque)	• FIFO (first in, first out) • add() adds to rear • remove()/poll() from front	Task queues, event processing, breadth-first search
Deque (e.g. ArrayDeque)	• Double-ended queue • add/remove at both ends • Acts as both queue and stack	Sliding windows, undo/redo, task scheduling

Example: Queue and Deque

```
Deque<String> dq = new ArrayDeque<>();
dq.addFirst("A");
dq.addLast("B");
dq.addLast("C");
System.out.println(dq.removeFirst());   // A
System.out.println(dq.removeLast());    // C
System.out.println(dq);                 // [B]
```

Output:

```
A
C
[B]
```

9 Performance & Behavior at a Glance

Quick comparison of common implementations

Collection	Allows Duplicates	Order	Typical Strengths	Common Weaknesses
ArrayList	Yes	Insertion	Fast random access	Slow middle inserts/removes
LinkedList	Yes	Insertion	Fast inserts/removes	Slower random access
HashSet	No	No order	Fast add/contions	No ordering guarantee
LinkedHashSet	No	Insertion	Keeps insertion order	Slightly higher memory cost
TreeSet	No	Sorted	Sorted elements	Slower add/contains
HashMap	No (keys)	No order	Fast put/get	No ordering guarantee
LinkedHashMap	No (keys)	Insertion	Keeps insertion order	Slightly higher memory cost
TreeMap	No (keys)	Sorted	Sorted keys	Slower put/get

10 Key Takeaways

Remember these collection guidelines

- **List:** ordered, allows duplicates. Use ArrayList for fast access, LinkedList for frequent middle modifications.
- **Set:** unique elements. Use HashSet (any order), LinkedHashSet (insertion order), or TreeSet (sorted).
- **Map:** key → value pairs with unique keys. Use HashMap (any order), LinkedHashMap (insertion order), or TreeMap (sorted).
- **Queue:** FIFO processing. **Deque:** add/remove from both ends.
- Choose the collection based on your required behavior (ordering, uniqueness, access pattern) and performance needs.
- Collections are part of the Java Collections Framework and work with generics for type safety.

11 Common Mistakes

Avoid these pitfalls

- Expecting a Set to keep insertion order (use LinkedHashSet).
- Forgetting that Sets do not allow duplicates.
- Using an ArrayList when frequent middle insertions/removals are needed.
- Confusing Map keys and values.
- Assuming HashMap or HashSet maintain a sorted order.
- Using a Queue when you need to add/remove from both ends (use a Deque).
- Forgetting to consider iteration order and performance requirements.

JAVA REFRESHER

Chapter 5 – Generics & PECS

Type Safety • Generic Classes & Methods • Bounded Types • Wildcards • PECS • Best Practices

Write Cleaner Safer, Reusable Code with Generics!

1 Why It Matters

Generics give you type safety, clearer code and reusable APIs.

- Catches errors at compile time (no ClassCastException).
- Makes intent clear and code easier to maintain.
- Lets you write reusable, type-safe collections and APIs.
- Eliminates unnecessary casts.

Without generics (unsafe)

```
List names = new ArrayList();
names.add("Java");
names.add(123);  // compiles, but risky

String s = (String) names.get(1);
// ClassCastException at runtime
```

With generics (safe)

```
List<String> names = new ArrayList<>();
names.add("Java");
// names.add(123);  // compile error
String s = names.get(0);   // safe
```

Only String elements allowed at compile time.

2 Generic Classes and Methods

Use type parameters to write reusable, type-safe code.

Generic Class Example

```
class Box<T>
{
    private T value;
    public void set(T value)
    {
        this.value = value;
    }

    public T get()
    {
        return value;
    }
}
```

T is a type parameter (placeholder for a real type, e.g., String, Integer).

Generic Method Example

```
public class Utils
{
  public static <T> T first(List<T> list)
  {
     return list.get(0);
  }
}
```

Usage

```
List<Integer> nums = List.of(1, 2, 3);
int firstNum = Utils.first(nums);
System.out.println(firstNum);
```

Output: 1

3 Type Parameters and Bounds

Restrict type parameters to specific types when needed.

Type Parameter Syntax

```
class SortedBox<T extends
      Comparable<T>>
{
   private T value;
   public void set(T value)
   {
      this.value = value;
   }
}
```

<T extends Number> allows T or its subtypes (Integer, Double…).

You can use multiple bounds.

Examples

```
public static <T extends Number>
 double sum(List<T> numbers)
 {
     double total = 0;
     for (T n : numbers)
     {
         total += n.doubleValue();
     }
     return total;
 }
```

Multiple Bounds

```
class SortableBox<T extends
Number & Comparable<T>> { ... }
```

4 Wildcards

Use wildcards when you don't know the exact type.

Wildcard	Meaning	Can Read?	Can Write?	Example
<?>	Unknown type	✔	✘	List<?> list
<? extends T>	Upper bound (T or subtype)	✔	✘	List<? extends Number> nums
<? super T>	Lower bound (T or supertype)	✔	✔	List<? super Integer> ints

Upper-bounded wildcard (read)

```
double sum = 0;
for (Number n : numbers)
{
    sum += n.doubleValue();
}
```

Lower-bounded wildcard (write)

```
public static void addIntegers(
   List<? super Integer> dest)
{
    dest.add(10);
    dest.add(20);
}
```

5 PECS – Producer Extends, Consumer Super

A simple rule to remember how to use wildcards.

Producer extends
Use ? extends T when you only **read** values (producer).

PECS

Consumer super
Use ? super T when you only **write** values (consumer).

Reading from a producer (extends)

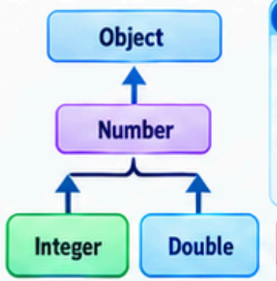
```
public static double total(
      List<? extends Number> nums)
{
   double sum = 0;
   for (Number n : nums)
   {
      sum += n.doubleValue();
   }
   return sum;
}
```

Writing to a consumer (super)

```
public static void addAllIntegers(
     List<? super Integer> dest)
{
    dest.add(1);
    dest.add(2);
    dest.add(3);
}
```

6 Type Relationships and Variance

Understanding why PECS works.



Subtyping

- Integer is a subtype of Number.
- Number is a subtype of Object.
- Generics are **invariant:** List<Integer> is not a subtype of List<Number>.

Why Invariance?

List<Integer> is not a List<Number> because it would allow adding non-Integers (e.g., 3.14).

Compile-time error

```
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;
// Error: incompatible types
```

7 Quick Examples & Output

Putting it all together.

Generic class usage

```
Box<String> box = new Box<>();
box.set("Hello");
System.out.println(box.get());
```

Output: Hello

Bounded type parameter

```
List<Integer> nums = List.of(1, 2, 3);
double sum = sum(nums);
System.out.println(sum);
```

Output: 6.0

PECS in action

```
List<Number> list = new ArrayList<>();
addAllIntegers(list);
System.out.println(list);
```

Output: [1, 2, 3]

8 Key Takeaways

Generics, Wildcards and PECS in practice.

- Generics provide compile-time type safety.
- Use type parameters <T> to create reusable classes and methods.
- Use bounds (extends) to restrict allowed types.
- Use wildcards when the exact type is unknown.
- Remember **PECS**: Producer Extends, Consumer Super.
- Generics are invariant (List<Integer> is not a List<Number>).
- Design clear, reusable and type-safe APIs with generics.

Common Mistakes

- Using raw types (e.g., List instead of List<T>).
- Overusing wildcards when a named type parameter is clearer.
- Trying to add elements to a <? extends …> collection.
- Confusing arrays and generics.
- Assuming List<Integer> is a List<Number>.

LESSON 1: Choosing List, Set, or Map

First understand the idea
Imagine that you have a bag of values. What do you need to do with it? Keep a sequence? Eliminate duplicates? Look up a value using a key? That question should drive your choice.

Important rules

• A List keeps order and duplicates, and gives access by index.

• A Set keeps unique elements; choose a specific implementation if iteration order matters.

• A Map is key → value lookup; keys are unique, values can repeat.

• Most collection APIs are interfaces, so declare variables as List, Set, Map rather than concrete types when possible.

Mental model
List = sequence; Set = unique membership; Map = keyed lookup.

Complete working code — 01_ListSetMapBasics.java

```
package refresher.chapter5;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

class ListSetMapBasics
{
    public static void main(String[] args)
    {
        // A List retains positional order and allows duplicates.
        List<String> tasks = new ArrayList<>();
        tasks.add("Load");
        tasks.add("Validate");
        tasks.add("Load");

        // A Set stores unique values. LinkedHashSet preserves insertion order.
        Set<String> uniqueTasks = new LinkedHashSet<>(tasks);

        // A Map associates a unique key with a value.
        Map<Integer, String> taskById = new LinkedHashMap<>();
        taskById.put(101, "Load");
        taskById.put(102, "Validate");
        taskById.put(101, "Reload"); // replaces value for key 101

        System.out.println("List: " + tasks);
        System.out.println("Set: " + uniqueTasks);
        System.out.println("Map: " + taskById);
        System.out.println("Item at index 1: " + tasks.get(1));

        System.out.println("Value at key 101: " + taskById.get(101));
    }
}
```

Actual program output

```
List: [Load, Validate, Load]
Set: [Load, Validate]
Map: {101=Reload, 102=Validate}
```

```
Item at index 1: Validate
Value at key 101: Reload
```

Connect the code to the concept

- Notice how putting key 101 twice in the Map replaces its earlier value.
- The Set contains only one Load, although the original list had two.
- LinkedHashSet and LinkedHashMap were chosen so their display order is predictable.

Check your understanding
Which operation would break if the data was a different collection type?

LESSON 2: Choosing collection implementations

First understand the idea
List, Set, and Map describe what a collection means. ArrayList, HashSet, and HashMap are actual implementations, each with different behavior. For most normal application code, choose the simplest implementation that satisfies the need.

Important rules

• ArrayList is the normal first choice for a List; random indexed access is fast.

• HashSet and HashMap prioritize efficient lookup but do not promise iteration order.

• LinkedHashSet/LinkedHashMap keep insertion order; TreeSet/TreeMap keep sorted order.

• ArrayDeque is a strong default for queue/deque behavior.

• LinkedList rarely wins for typical business code; inserting in the middle still requires locating the position.

Mental model
Remember the behavior you need before choosing syntax.

Complete working code — 02_ImplementationsAndIteration.java

```
package refresher.chapter5;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.TreeSet;

class ImplementationsAndIteration
{
    public static void main(String[] args)
    {
        List<Integer> scores = new ArrayList<>(List.of(70, 80, 90, 80));
        Iterator<Integer> iterator = scores.iterator();
        while (iterator.hasNext())
        {
            if (iterator.next() < 80)
            {
                iterator.remove(); // correct removal during iterator traversal
            }
        }
        System.out.println("After remove: " + scores);

        Queue<String> jobs = new ArrayDeque<>();
        jobs.add("A");
        jobs.add("B");
        System.out.println("Queue FIFO: " + jobs.remove() + ", " + jobs.remove());

        System.out.println("Sorted set: " + new TreeSet<>(List.of(8, 3, 8, 1)));

        // HashMap ordering is not guaranteed; LinkedHashMap preserves insertion order.
        Map<Integer, String> names = new LinkedHashMap<>();
        names.put(2, "Bob");
        names.put(1, "Alice");
        for (Map.Entry<Integer, String> entry : names.entrySet())
        {
```

```
            System.out.println(entry.getKey() + " = " + entry.getValue());
        }
    }
}
```

Actual program output

```
After remove: [80, 90, 80]
Queue FIFO: A, B
Sorted set: [1, 3, 8]
2 = Bob
1 = Alice
```

Connect the code to the concept

- The iterator.remove() example shows the supported way to remove while iterating.
- ArrayDeque removes the first inserted job: FIFO.
- TreeSet returns numbers in sorted order with duplicates removed.

Check your understanding
Can you explain the printed output without running the program?

LESSON 3: Generics: controlling what types are allowed

First understand the idea
Generics let you say what kind of values belong in a collection or box. The compiler checks the types before the program runs. This makes the intent explicit and prevents many runtime ClassCastException failures.

Important rules

- List means a List whose elements must be Strings.
- Generic classes use a placeholder type such as T.
- Generic methods may declare their own T; they are different from generic classes.
- Java generic types are invariant: List is not a List.
- At runtime most generic arguments are erased; generics mainly provide compile-time type safety.

Mental model
Remember the behavior you need before choosing syntax.

Complete working code — 03_GenericsMethodsPECS.java

```
package refresher.chapter5;

import java.util.ArrayList;
import java.util.List;

class GenericsMethodsPECS
{
    static class Box<T>
    {
        private final T value;

        Box(T value)
        {
            this.value = value;
        }

        T get()
        {
            return value;
        }
    }

    static <T> T first(List<T> values)
    {
        return values.get(0);
    }

    // Producer Extends: read elements as Number.
    static double sum(List<? extends Number> numbers)
    {
        double total = 0;
        for (Number number : numbers)

        {
            total += number.doubleValue();
        }
        return total;
    }

    // Consumer Super: safely add Integer elements.
    static void addDefaults(List<? super Integer> target)
```

```
    {
        target.add(10);
        target.add(20);
    }

    public static void main(String[] args)
    {
        Box<String> box = new Box<>("Hello");
        System.out.println("Box: " + box.get());
        System.out.println("First: " + first(List.of("A", "B")));
        System.out.println("Sum: " + sum(List.of(1, 2, 3)));
        List<Number> output = new ArrayList<>();
        addDefaults(output);
        System.out.println("Consumer: " + output);
        // List<Number> invalid = new ArrayList<Integer>(); // compile error: invariant
    }
}
```

Actual program output

```
Box: Hello
First: A
Sum: 6.0
Consumer: [10, 20]
```

Connect the code to the concept

- Box works with different types without duplicating its implementation.
- The compiler infers T in first(List.of("A", "B")).
- Try changing a List element to an integer and observe the compile error.

Check your understanding
Why does this data structure or type constraint fit?

LESSON 4: PECS: Producer Extends, Consumer Super

First understand the idea
A common confusion is why List cannot be passed to a method expecting List. Java disallows this because the method could insert a Double into that Integer list. Wildcards give more flexible, safe contracts.

Important rules

- ? extends Number: read Numbers from the input; do not add arbitrary Numbers to it.
- ? super Integer: write Integers to the destination; reading elements yields Object.
- PECS describes how your method uses the collection, not what kind of class the collection is.
- You rarely need wildcards everywhere: use them at APIs where generic flexibility helps.

Mental model
Remember the behavior you need before choosing syntax.

Complete working code — 03_GenericsMethodsPECS.java

```
package refresher.chapter5;

import java.util.ArrayList;
import java.util.List;

class GenericsMethodsPECS
{
    static class Box<T>
    {
        private final T value;

        Box(T value)
        {
            this.value = value;
        }

        T get()
        {
            return value;
        }
    }

    static <T> T first(List<T> values)
    {
        return values.get(0);
    }

    // Producer Extends: read elements as Number.
    static double sum(List<? extends Number> numbers)
    {
        double total = 0;
        for (Number number : numbers)

        {
            total += number.doubleValue();
        }
        return total;
    }

    // Consumer Super: safely add Integer elements.
    static void addDefaults(List<? super Integer> target)
    {
        target.add(10);
```

```
        target.add(20);
    }

    public static void main(String[] args)
    {
        Box<String> box = new Box<>("Hello");
        System.out.println("Box: " + box.get());
        System.out.println("First: " + first(List.of("A", "B")));
        System.out.println("Sum: " + sum(List.of(1, 2, 3)));
        List<Number> output = new ArrayList<>();
        addDefaults(output);
        System.out.println("Consumer: " + output);
        // List<Number> invalid = new ArrayList<Integer>(); // compile error: invariant
    }
}
```

Actual program output

```
Box: Hello
First: A
Sum: 6.0
Consumer: [10, 20]
```

Connect the code to the concept

- sum(List) can accept List and List.
- addDefaults(List) can add Integer values to List.
- Uncomment the invalid assignment to see invariance enforced by the compiler.

Check your understanding
Which operation would break if the data was a different collection type?

LESSON 5: Equality and hashing: how Sets and Maps recognize keys

First understand the idea
A HashSet or HashMap cannot guess which two business objects should count as the same. equals() and hashCode() determine that relationship. If you get the relationship wrong, duplicate checks and map lookup can behave unexpectedly.

Important rules

• Equal objects must return equal hash codes.

• Unequal objects may have colliding hash codes; equality still decides the match.

• Choose stable identity fields for keys; avoid mutating fields that affect equality after insertion.

• Java records automatically implement component-based equals() and hashCode(); custom domain identity may differ.

Mental model
Remember the behavior you need before choosing syntax.

Complete working code — 04_EqualityAndHashing.java

```
package refresher.chapter5;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Objects;
import java.util.Set;

class EqualityAndHashing
{
    static final class Customer
    {
        final int id;
        final String name;

        Customer(int id, String name)
        {
            this.id = id;
            this.name = name;
        }

        @Override
        public boolean equals(Object other)
        {
            if (this == other) return true;
            if (!(other instanceof Customer customer)) return false;
            return id == customer.id; // business identity is id
        }

        @Override
        public int hashCode()
        {

            return Objects.hash(id);
        }
    }

    public static void main(String[] args)
    {
        Customer a = new Customer(7, "Anita");
        Customer b = new Customer(7, "Anita Updated");
```

```
        System.out.println("equals: " + a.equals(b));
        System.out.println("Set size: " + new HashSet<>(java.util.Arrays.asList(a, b)).size());
        Map<Customer, String> status = new HashMap<>();
        status.put(a, "ACTIVE");
        System.out.println("Lookup by equal key: " + status.get(b));
    }
}
```

Actual program output

```
equals: true
Set size: 1
Lookup by equal key: ACTIVE
```

Connect the code to the concept

- Two Customer objects with the same ID are equal even though their names differ.
- The Set retains one of the two equal Customer values.
- The Map successfully looks up a value using a different but equal key.

Check your understanding
Can you explain the printed output without running the program?

LESSON 6: Map methods that eliminate repetitive loops

First understand the idea
A Map often groups data by key or accumulates totals. You can write the logic with containsKey(), but methods such as merge(), computeIfAbsent(), and getOrDefault() express common intent more clearly.

Important rules

• merge(key, value, operation) combines a new value with an existing value.

• computeIfAbsent(key, function) creates a value only when absent or mapped to null.

• getOrDefault provides a fallback without putting that fallback in the Map.

• Be careful with null keys or values: support depends on the implementation.

Mental model
Remember the behavior you need before choosing syntax.

Complete working code — 05_MapsGroupingAndDefaults.java

```
package refresher.chapter5;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

class MapsGroupingAndDefaults
{
    public static void main(String[] args)
    {
        List<String> departments = List.of("IT", "HR", "IT", "SALES", "IT");
        Map<String, Integer> counts = new LinkedHashMap<>();
        for (String department : departments)
        {
            counts.merge(department, 1, Integer::sum);
        }
        System.out.println("Counts: " + counts);

        Map<String, List<String>> employees = new LinkedHashMap<>();
        employees.computeIfAbsent("IT", unused -> new ArrayList<>()).add("Alice");
        employees.computeIfAbsent("IT", unused -> new ArrayList<>()).add("Bob");
        employees.computeIfAbsent("HR", unused -> new ArrayList<>()).add("Cara");
        System.out.println("Employees: " + employees);

        System.out.println("Unknown count: " + counts.getOrDefault("LEGAL", 0));
    }
}
```

Actual program output

```
Counts: {IT=3, HR=1, SALES=1}
Employees: {IT=[Alice, Bob], HR=[Cara]}
Unknown count: 0
```

Connect the code to the concept

• merge produces correct counts with no explicit containsKey branch.

• computeIfAbsent creates a List for a department only the first time.

• getOrDefault returns 0 for a department not in the Map.

Check your understanding
Why does this data structure or type constraint fit?

LESSON 7: Mutable, unmodifiable, and immutable collections

First understand the idea
These words look similar but describe different behavior. An unmodifiable view stops a caller from changing data through the view, but someone else may still change the underlying list. A snapshot copies the elements at a moment in time.

Important rules

• Collections.unmodifiableList(list) is a read-only view, not a defensive copy.

• List.copyOf(list) creates an unmodifiable snapshot of elements.

• List.of(...) produces an unmodifiable List and rejects null elements.

• The copy is shallow: mutable elements inside are not deep-copied.

• Collections are generally not thread-safe merely because they are unmodifiable.

Mental model
Remember the behavior you need before choosing syntax.

Complete working code — 06_CollectionSafetyAndCopies.java

```
package refresher.chapter5;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class CollectionSafetyAndCopies
{
    public static void main(String[] args)
    {
        List<String> original = new ArrayList<>(List.of("A", "B"));
        List<String> view = Collections.unmodifiableList(original);
        List<String> snapshot = List.copyOf(original);

        original.add("C");
        System.out.println("Original: " + original);
        System.out.println("Unmodifiable view: " + view); // reflects changes in original
        System.out.println("Immutable snapshot: " + snapshot);

        // view.add("D");     // UnsupportedOperationException
        // snapshot.add("D"); // UnsupportedOperationException
        // A shallow snapshot does NOT clone mutable elements inside it.
    }
}
```

Actual program output

```
Original: [A, B, C]
Unmodifiable view: [A, B, C]
Immutable snapshot: [A, B]
```

Connect the code to the concept

• After original.add("C"), the view contains C but the snapshot does not.

• Both the view and snapshot reject add().

• A snapshot of a list of mutable objects still contains references to those objects.

Check your understanding
Which operation would break if the data was a different collection type?

LESSON 8: Putting everything together: inventory processing

First understand the idea
The previous lessons may feel separate until you see one program use all of them. The inventory example keeps a product list, creates a SKU lookup map, finds distinct categories, summarizes stock, sorts products, and uses a queue for restocking.

Important rules

- List maintains the original sequence and allows sorting a separate copy.
- Map gives direct SKU lookup.
- Set captures each category once.
- Map merges quantities by category.
- Queue represents pending restock work in FIFO order.

Mental model
Remember the behavior you need before choosing syntax.

Complete working code — 07_InventoryMasterExample.java

```
package refresher.chapter5;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

class InventoryMasterExample
{
    // Records provide value-based equals/hashCode for immutable data components.
    record Product(String sku, String category, int quantity)
    {
    }

    static <T> void printList(String label, List<T> values)
    {
        System.out.println(label + ": " + values);
    }

    public static void main(String[] args)
    {
        List<Product> products = List.of(
            new Product("P1", "BOOK", 3),
            new Product("P2", "TECH", 8),
            new Product("P3", "BOOK", 5),
            new Product("P4", "TECH", 1)
        );

        // Map: direct lookup by unique SKU.
        Map<String, Product> bySku = new LinkedHashMap<>();
        for (Product product : products)
        {
            bySku.put(product.sku(), product);
        }
```

```
        // Set: which categories exist, without duplicates.
        Set<String> categories = new LinkedHashSet<>();
        for (Product product : products)
        {
            categories.add(product.category());
        }

        // Map with merge: total quantity per category.
        Map<String, Integer> categoryTotals = new LinkedHashMap<>();
        for (Product product : products)
        {
            categoryTotals.merge(product.category(), product.quantity(), Integer::sum);
        }

        // List: preserve duplicates/order and sort a copy.
        List<Product> stockAscending = new ArrayList<>(products);
        stockAscending.sort(Comparator.comparingInt(Product::quantity));

        // Queue: FIFO restock requests.
        Queue<String> restock = new ArrayDeque<>();
        for (Product product : stockAscending)
        {
            if (product.quantity() < 4)
            {

                restock.add(product.sku());
            }
        }

        printList("Products by stock", stockAscending);
        System.out.println("Categories: " + categories);
        System.out.println("Totals: " + categoryTotals);
        System.out.println("SKU P2: " + bySku.get("P2"));
        System.out.println("Restock first: " + restock.remove());
        System.out.println("Restock remaining: " + restock);
    }
}
```

Actual program output

```
Products by stock: [Product[sku=P4, category=TECH, quantity=1], Product[sku=P1, category=BOOK, quantity=
Categories: [BOOK, TECH]
Totals: {BOOK=8, TECH=9}
SKU P2: Product[sku=P2, category=TECH, quantity=8]
Restock first: P4
Restock remaining: [P1]
```

Connect the code to the concept

- The output shows the correct sorted stock order and category totals.
- The restock queue captures low-stock product SKUs in ascending stock order.
- Ask which two data structures could be replaced without changing business meaning, and which could not.

Check your understanding
Can you explain the printed output without running the program?

Quick reference: choose by behavior

Requirement	Use first	Important qualification
Indexed ordered sequence; duplicates allowed	ArrayList	Insertion in middle shifts elements
Unique membership	HashSet	No guaranteed order
Unique, predictable insertion order	LinkedHashSet	Extra bookkeeping
Key to value lookup	HashMap	No guaranteed order
Map in insertion order	LinkedHashMap	Useful for stable reports
Sorted keys or values	TreeMap / TreeSet	Comparator or natural ordering
FIFO task processing	ArrayDeque	Avoid nulls
Read varied subtype lists	? extends T	Producer
Write subtype values into list	? super T	Consumer

Five habits worth developing
Declare variables using interfaces. Choose explicit ordering when it matters. Keep hash keys stable. Use generic types instead of raw collections. Prefer a simple loop when it communicates intent more clearly than a clever one-liner.

Chapter 5 — Depth 9 Expansion

Collections and Generics: deeper mechanics, practical debugging, complete Java 17 programs, and worked exercises.

How to use this expansion: The original concept-first lessons and three glossy infographics remain in the main chapter. The pages that follow strengthen the material that was too brief in the previous edition: performance, iteration, generics, PECS, equality and hashing, maps, and safe copying.

1. Choosing a collection is a behavior decision

A **List** is useful when duplicates and positions matter. A **Set** models uniqueness. A **Map** associates unique keys with values. Those are contracts, not particular storage mechanisms. Choose an implementation only after you know your requirements for ordering, speed, null handling, and mutation.

Implementation	Typical lookup	Ordering	Best fit
ArrayList	get(index): O(1)	Index / insertion	Read-heavy lists
LinkedList	get(index): O(n)	Index / insertion	Deque operations; seldom ideal as List
HashSet	contains: O(1)*	Unspecified	Fast uniqueness checks
LinkedHashSet	contains: O(1)*	Insertion order	Predictable unique iteration
TreeSet	contains: O(log n)	Sorted	Range and navigation queries
HashMap	get(key): O(1)*	Unspecified	General key/value lookup
TreeMap	get(key): O(log n)	Sorted keys	Sorted and range-based maps

* Typical average complexity for healthy hashing. Worst-case behavior, memory cost, and constant factors can matter. LinkedList is rarely a good replacement for ArrayList just because you insert or remove values: locating an arbitrary position still costs O(n).

What happens inside a HashMap?

Java calculates the key hash, identifies a bucket, and compares keys within the bucket using equality. Collisions are normal: different keys may land in one bucket. Java can use tree bins under specific collision conditions. Resizing reallocates the table when capacity thresholds are crossed; it is not free, but is amortized across many insertions.

2. Iteration and structural modification

A foreach loop is backed by an iterator. Many ordinary mutable collection iterators are fail-fast: if you structurally modify the collection outside the iterator while traversing it, a ConcurrentModificationException may be thrown. This is a best-effort bug detector, not a thread-safety guarantee.

Use **Iterator.remove()**, **removeIf()**, or carefully chosen collection APIs for safe removal. Do not call list.remove(element) from inside a foreach loop over that same list. Not every collection has fail-fast iterators: concurrent collections can have weakly consistent traversal semantics.

Reading clue: when you see List<T>, ask whether order and duplicates matter. When you see HashSet<T>, ask what equals/hashCode means. When you see TreeMap<K,V>, ask what comparator orders keys.

Ordering is not sorting

LinkedHashMap and LinkedHashSet preserve insertion order, while TreeMap and TreeSet sort by natural ordering or a supplied comparator. HashMap and HashSet promise neither insertion nor sorted iteration order. Never rely on the particular order you happen to observe in a test.

Worked prediction

A LinkedHashSet receives B, A, B, C. What does it contain? Answer: [B, A, C]. B is not stored twice, and its original position stays the same. A TreeSet with these same values would iterate [A, B, C].

3. Generics: what the compiler knows and erases

A generic type parameter is a placeholder for a type. The compiler checks assignments and method calls using that parameter. Most generic type information is erased from ordinary runtime objects, so List<String> and List<Integer> are both commonly backed by the same ArrayList runtime class. Generic methods can be declared independently of generic classes.

Invariance is intentional

Although Integer extends Number, List<Integer> does not extend List<Number>. Otherwise code could put Double into a List<Integer> by viewing it as a List<Number>. A wildcard describes controlled flexibility without breaking type safety.

```
// Does not compile:
List<Integer> ints = new ArrayList<>();
List<Number> numbers = ints; // NOT SAFE
// If allowed, numbers.add(3.14) would corrupt ints.
```

Bounds and inference

A method such as <T extends Comparable<? super T>> T maximum(List<T>) accepts types that can be compared under the bound. A generic bound may include interfaces. Java usually infers T from arguments, but at times an explicit type argument clarifies ambiguous calls.

Type erasure limitations

You cannot write new T[10], use instanceof List<String> to test a parameterized runtime type, or overload two methods that erase to the same signature. Raw types disable much compiler checking and can cause heap pollution and delayed ClassCastException.

Important distinction: arrays are covariant but enforce their component type at runtime; generic lists are invariant and enforce type rules primarily during compilation. That difference explains why Object[] can refer to String[] yet storing an Integer triggers ArrayStoreException.

4. PECS: reading versus writing

PECS means **Producer Extends, Consumer Super**. A producer provides values to your code; a consumer accepts values from your code. It describes how your method uses a collection—not a permanent label on the collection itself.

```
static double total(List<? extends Number> source)
{
    double sum = 0;
    for (Number value : source)
    {
        sum += value.doubleValue(); // reading safely as Number
    }
    return sum;
}
static void addScores(List<? super Integer> destination)
{
    destination.add(75); // safe: Integer can go into any supertype list
}
```

Why can I read Number but not add Number?

List<? extends Number> could actually refer to List<Integer>. Reading yields at least a Number, but adding an arbitrary Number could put Double into an Integer list. You may generally add null, not meaningful Number instances. For List<? super Integer>, writing Integer is safe, while reading gives only Object without further type information.

PECS edge cases

Choose List<T> when your method both reads and writes values of T, because neither extends nor super captures both abilities cleanly. The wildcard ? alone permits reading as Object but usually prevents adding anything except null.

Exercise: copy data from List<Integer> into List<Number>. The source should be List<? extends T>; the destination should be List<? super T>. That is the same principle used by Collections.copy().

5. Equality and hashing: why lookups can fail

HashSet and HashMap depend on two agreements. Equal objects must have equal hash codes; unequal objects may still have equal hash codes. equals() must be reflexive, symmetric, transitive, consistent, and false for null. Java records normally provide value-based equals/hashCode for their components, making immutable IDs particularly convenient.

Mutable keys are dangerous

Suppose a HashMap stores a key while its code is "X". The map selects a bucket based on hashCode(). If the same key object changes to "Y", a later get(key) may search another bucket. The entry is still present, but lookup can fail. Keep hash-relevant fields immutable while the object acts as a map key.

Practical rule: use stable identity fields for equals() and hashCode(). Consider records for small immutable value keys, and be careful with ORM entities whose IDs may be assigned after persistence.

Worked trace

1. Put key X into its bucket. 2. Change X to Y. 3. get(key) hashes Y. 4. It searches a bucket that may not contain the entry. 5. get() returns null. This is not a corrupted HashMap; it is a broken key contract.

6. Maps: choosing the correct mutation method

Map code looks compact, but each operation has a different contract. Learning the difference makes unfamiliar business code much easier to read and debug.

putIfAbsent(k,v) — Associates v if key has no value or is mapped to null.

computeIfAbsent(k,f) — Computes a new value only when the existing value is absent/null; null result means no mapping added.

computeIfPresent(k,f) — Remaps a key only when it has a non-null existing value. Returning null removes it.

compute(k,f) — Runs the function even if the key is absent; a null result removes or leaves absent.

merge(k,v,f) — Uses v if absent/null; otherwise combines old and v. A null remapping result removes the key.

replace(k,v) — Replaces only when the key already has a mapping.

A common grouping idiom

```
Map<String, List<String>> peopleByTeam = new HashMap<>();
peopleByTeam.computeIfAbsent("Engineering", key -> new ArrayList<>())
            .add("Alice");
peopleByTeam.computeIfAbsent("Engineering", key -> new ArrayList<>())
            .add("Charlie");
System.out.println(peopleByTeam);
```

Null semantics need deliberate thought

A HashMap permits a null value; a ConcurrentHashMap does not. Because get(k) can return null for either “missing” or “mapped to null” in maps permitting nulls, use containsKey when the distinction matters. Some Map default methods explicitly treat a null current value as absent.

7. Unmodifiable is not deep immutability

List.copyOf(original) creates an unmodifiable snapshot of the list structure. It does not clone the objects inside. If elements themselves are mutable, modifying an element through another reference changes what the copied list appears to contain. Collections.unmodifiableList(original) is different: it is a read-only view reflecting structural changes made through original.

Useful distinctions

new ArrayList<>(source) is a new mutable list with the same element references. **List.copyOf(source)** is unmodifiable and rejects null elements. **Collections.unmodifiableList(source)** is a view, not a snapshot. **List.of(...)** creates unmodifiable lists with no nulls. None automatically deep-copy mutable elements.

Thread safety is separate

Unmodifiable does not necessarily mean thread-safe if referenced elements are mutable. ConcurrentHashMap and CopyOnWriteArrayList solve different concurrency problems. Collections.synchronizedList requires external synchronization during iteration; concurrent structures have their own access and iteration rules.

Practical design advice: prefer immutable records for DTO-like values when appropriate, List.copyOf for ownership boundaries, and make deep copies only when mutable elements must be isolated.

Worked lab — Iterator removal and ordering

This example is included as a complete, separately runnable Java file. Read the behavior first, predict its output, then compare with the verified run.

• ArrayList provides fast positional access but iteration is the normal way to walk every element.

• Iterator.remove removes the last element returned by next, safely updating iterator state.

• LinkedHashSet preserves insertion order; TreeSet provides sorted navigation like ceiling.

Prediction and debugging check

What if you used items.remove("B") inside for-each? It can trigger ConcurrentModificationException. Use removeIf or Iterator.remove instead.

Verified console output

```
After iterator.remove: [A, C, D]
Insertion order: [B, A, C]
Sorted: [3, 7, 9]
Smallest >= 5: 7
```

Complete runnable code — CollectionMechanics.java

```
import java.util.*;
public class CollectionMechanics
{
    public static void main(String[] args)
    {
        List<String> items = new ArrayList<>(List.of("A", "B", "C", "D"));
        Iterator<String> iterator = items.iterator();
        while (iterator.hasNext())
        {
            if (iterator.next().equals("B"))
            {
                iterator.remove(); // legal structural removal during iteration
            }
        }
        System.out.println("After iterator.remove: " + items);
        Set<String> ordered = new LinkedHashSet<>(List.of("B", "A", "C"));
        System.out.println("Insertion order: " + ordered);
        NavigableSet<Integer> sorted = new TreeSet<>(List.of(9, 3, 7));
        System.out.println("Sorted: " + sorted);
        System.out.println("Smallest >= 5: " + sorted.ceiling(5));
    }
}
```

What to inspect in IntelliJ

Set a breakpoint near the main collection operation and inspect the collection before and after it. Compare the compile-time declared type against the actual runtime class. Then change one line and predict whether the behavior, ordering, or compilation result changes.

Worked lab — PECS and bounded generic methods

This example is included as a complete, separately runnable Java file. Read the behavior first, predict its output, then compare with the verified run.

• total reads from a producer and works with List or List.

• addScores writes Integer values into List, List, or List.

• maximum shows an upper bound involving Comparable and a safely inferred type.

Prediction and debugging check

Why is List n = integers illegal? Because an insertion through n could violate the real List element type.

Verified console output

```
Producer sum = 30.0
Consumer = [75, 90]
Maximum = C
```

Complete runnable code — GenericsVariancePECS.java

```
import java.util.*;
public class GenericsVariancePECS
{
    static double total(List<? extends Number> source)
    {
        double sum = 0;
        for (Number number : source)
        {
            sum += number.doubleValue();
        }
        return sum;
    }
    static void addScores(List<? super Integer> destination)
    {
        destination.add(75);
        destination.add(90);
    }
    static <T extends Comparable<? super T>> T maximum(List<T> data)
    {
        T result = data.get(0);
        for (T value : data)
        {
            if (value.compareTo(result) > 0)
            {
                result = value;
            }
        }
        return result;
    }
    public static void main(String[] args)
    {
        List<Integer> integers = List.of(5, 10, 15);
        List<Number> numbers = new ArrayList<>();
        addScores(numbers);
        System.out.println("Producer sum = " + total(integers));
        System.out.println("Consumer = " + numbers);
        System.out.println("Maximum = " + maximum(List.of("A", "C", "B")));
        // INVALID: List<Number> x = integers; // generics invariant
        // INVALID: List<? extends Number> x = integers; x.add(4);
    }
}
```

What to inspect in IntelliJ

Set a breakpoint near the main collection operation and inspect the collection before and after it. Compare the compile-time declared type against the actual runtime class. Then change one line and predict whether the behavior, ordering, or compilation result changes.

Worked lab — Stable identity and HashMap lookup

This example is included as a complete, separately runnable Java file. Read the behavior first, predict its output, then compare with the verified run.

• Record keys are value-based and stable when their components are immutable.

• The mutable key works before mutation but can fail lookup after a hash-relevant field changes.

• Changing a key after insertion can make get fail even using that exact object.

Prediction and debugging check

Repair: remove and reinsert before any key mutation, but a better design uses immutable key state.

Verified console output

```
Record lookup = Alice
Before mutation = value
After mutation = null
```

Complete runnable code — EqualityMutableKeys.java

```
import java.util.*;
public class EqualityMutableKeys
{
    record CustomerId(String value) {}
    static final class MutableKey
    {
        String code;
        MutableKey(String code) { this.code = code; }
        @Override public int hashCode() { return code.hashCode(); }
        @Override public boolean equals(Object obj)
        {
            return obj instanceof MutableKey other && code.equals(other.code);
        }
    }
    public static void main(String[] args)
    {
        Map<CustomerId, String> safe = new HashMap<>();
        safe.put(new CustomerId("C1"), "Alice");
        System.out.println("Record lookup = " + safe.get(new CustomerId("C1")));
        MutableKey key = new MutableKey("X");
        Map<MutableKey, String> unsafe = new HashMap<>();
        unsafe.put(key, "value");
        System.out.println("Before mutation = " + unsafe.get(key));
        key.code = "Y"; // changes the hash after insertion
        System.out.println("After mutation = " + unsafe.get(key));
    }
}
```

What to inspect in IntelliJ

Set a breakpoint near the main collection operation and inspect the collection before and after it. Compare the compile-time declared type against the actual runtime class. Then change one line and predict whether the behavior, ordering, or compilation result changes.

Worked lab — Map compute, merge, replace, and safe removal

This example is included as a complete, separately runnable Java file. Read the behavior first, predict its output, then compare with the verified run.

- The first red merge sets 1; the second makes 3, then computeIfPresent changes it to 30.
- computeIfAbsent creates the blue value from its key length, later replaced by 8.
- merge with a function returning null removes green.

Prediction and debugging check

Predict: What does computeIfAbsent do if a key already has a non-null value? It does not call its function.

Verified console output

```
After updates = {blue=8, green=4, red=30}
After null merge = {blue=8, red=30}
After removal = {blue=8}
```

Complete runnable code — MapMutation.java

```
import java.util.*;
public class MapMutation
{
    public static void main(String[] args)
    {
        Map<String, Integer> counts = new TreeMap<>();
        counts.merge("red", 1, Integer::sum);
        counts.merge("red", 2, Integer::sum);
        counts.computeIfAbsent("blue", key -> key.length());
        counts.computeIfPresent("red", (key, value) -> value * 10);
        counts.compute("green", (key, value) -> value == null ? 4 : value + 1);
        counts.replace("blue", 8);
        System.out.println("After updates = " + counts);
        counts.merge("green", 5, (oldValue, newValue) -> null);
        System.out.println("After null merge = " + counts);
        counts.entrySet().removeIf(entry -> entry.getValue() > 20);
        System.out.println("After removal = " + counts);
    }
}
```

What to inspect in IntelliJ

Set a breakpoint near the main collection operation and inspect the collection before and after it. Compare the compile-time declared type against the actual runtime class. Then change one line and predict whether the behavior, ordering, or compilation result changes.

Worked lab — Shallow copies and unmodifiable snapshots

This example is included as a complete, separately runnable Java file. Read the behavior first, predict its output, then compare with the verified run.

• All three lists hold the same Item reference, so name mutation is visible in each list.

• The snapshot does not see an appended second item because its structure is independent.

• Unmodifiable means the list cannot be changed through that reference; element objects can still change.

Prediction and debugging check

How to isolate Item content? Create a copy of each element, or use immutable element types.

Verified console output

```
Original = [new]
Snapshot = [new]
Shallow = [new]
Snapshot size = 1
```

Complete runnable code — CopySemantics.java

```
import java.util.*;
public class CopySemantics
{
    static final class Item
    {
        String name;
        Item(String name) { this.name = name; }
        @Override public String toString() { return name; }
    }
    public static void main(String[] args)
    {
        Item item = new Item("old");
        List<Item> original = new ArrayList<>();
        original.add(item);
        List<Item> snapshot = List.copyOf(original);
        List<Item> shallow = new ArrayList<>(original);
        item.name = "new";
        System.out.println("Original = " + original);
        System.out.println("Snapshot = " + snapshot);
        System.out.println("Shallow = " + shallow);
        original.add(new Item("second"));
        System.out.println("Snapshot size = " + snapshot.size());
        // snapshot.add(new Item("x")); // UnsupportedOperationException
    }
}
```

What to inspect in IntelliJ

Set a breakpoint near the main collection operation and inspect the collection before and after it. Compare the compile-time declared type against the actual runtime class. Then change one line and predict whether the behavior, ordering, or compilation result changes.

Worked lab — Runtime representation and invalid generic operations

This example is included as a complete, separately runnable Java file. Read the behavior first, predict its output, then compare with the verified run.

• Two differently parameterized ArrayLists have the same runtime class.

• A String[] viewed as Object[] is still checked against String at runtime.

• Parameterized instanceof and new T[] are restricted because type parameters are erased.

Prediction and debugging check

Which catches incorrect values earlier: List or Object[] holding String[]? The generic List checks at compile time; the array can fail at runtime.

Verified console output

```
Runtime class = ArrayList
Runtime class = ArrayList
ArrayStoreException caught
```

Complete runnable code — ErasureAndArrays.java

```
import java.util.*;
public class ErasureAndArrays
{
    static <T> void printType(List<T> values)
    {
        System.out.println("Runtime class = " + values.getClass().getSimpleName());
    }
    public static void main(String[] args)
    {
        List<String> names = new ArrayList<>(List.of("A"));
        List<Integer> scores = new ArrayList<>(List.of(1));
        printType(names);
        printType(scores);
        Object[] a = new String[1];
        try
        {
            a[0] = 99; // arrays enforce their component type at runtime
        }
        catch (ArrayStoreException exception)
        {
            System.out.println("ArrayStoreException caught");
        }
        // INVALID: new T[10], instanceof List<String>, List<String>[10]
    }
}
```

What to inspect in IntelliJ

Set a breakpoint near the main collection operation and inspect the collection before and after it. Compare the compile-time declared type against the actual runtime class. Then change one line and predict whether the behavior, ordering, or compilation result changes.

8. Applied exercises with worked answers

Question: A HashMap key has an equals method but no matching hashCode. Why is lookup unreliable?

Worked answer: Equal keys can hash into different buckets, breaking the hash contract. Override both consistently or use a record.

Question: You need users in sign-up order without duplicates. Which collection?

Worked answer: LinkedHashSet. It enforces uniqueness while maintaining insertion order.

Question: You need the 10 smallest scores repeatedly and sorted. Which set helps?

Worked answer: TreeSet supports sorted iteration and navigational queries, but preserves only unique scores. If duplicates matter, use another structure.

Question: Can you add 3.14 to List?

Worked answer: No. It could actually reference List. Reading as Number is safe; adding arbitrary Number is not.

Question: Can List read an element as Integer?

Worked answer: Not generally; the declared bound only promises Object on retrieval. You can safely write Integer values.

Question: What does merge(k,v,f) do when f returns null?

Worked answer: It removes the mapping.

Question: Does List.copyOf of mutable objects deep-copy those objects?

Worked answer: No. It creates an unmodifiable list structure; elements are the same references.

Question: Why does iterating HashMap look sorted in one test?

Worked answer: Its iteration order is unspecified and can change with hash codes, resizing or implementation details. Never rely on it.

9. Final decision checklist

• Need indexed access and duplicates? Start with ArrayList.

• Need fast uniqueness? HashSet; predictable insertion order? LinkedHashSet.

• Need sorted unique data or navigable operations? TreeSet.

• Need key/value lookup? HashMap; sorted keys? TreeMap.

• Need generic flexibility? Apply PECS based on producer versus consumer role.

• Need stable map keys? Design immutable equals/hashCode identity.

• Need safe mutation during iteration? Use Iterator.remove or removeIf.

• Need a defensive list snapshot? List.copyOf, but review element mutability.

• Need concurrent access? Select a concurrent collection based on semantics.

Depth target: The expanded edition now includes conceptual foundations, typical complexity, inner workings, full code, verified console output, debugging exercises, and practical selection guidance. Rarely useful implementation trivia has been intentionally omitted.